

АННОТАЦИЯ

Исполнитель: студент группы ЭФМО-02-26 Бондарь А.Р.

Общее количество страниц: 41.

Работа включает в себя 21 рисунок, 1 таблица, 6 приложений. Список источников информации включает 16 позиций.

Ключевые слова: интеллектуальный поиск, PDF-документы, полнотекстовый поиск, семантический поиск, векторные представления, микросервисная архитектура, Go, Python, PostgreSQL, pgvector, Redis, MinIO, Docker Compose.

В настоящей работе представлена разработка системы интеллектуального поиска по PDF-документам, объединяющей классический полнотекстовый и семантический векторный поиск. Система реализована на основе микросервисной архитектуры с использованием современных технологий: Go, Python, PostgreSQL с расширением pgvector, Redis, MinIO и Docker Compose.

СОДЕРЖАНИЕ

Аннотация	1
Введение	4
1 Обзор существующих решений.....	6
1.1 Традиционные полнотекстовые движки	6
1.2 Системы семантического (векторного) поиска.....	8
1.3 Интегрированные платформы для поиска по документам	9
1.4 Анализ существующих решений	10
1.5 Интерпретация результатов сравнения	10
2 Описание программируемой системы	12
2.1 Функциональные требования.....	12
2.2 Нефункциональные требования	13
2.3 Архитектура системы	13
2.3.1 Основные компоненты системы	14
2.3.2 Взаимодействие контейнеров	15
2.3.3 Потоки данных	15
2.3.4 Диаграмма контейнеров.....	15
2.4 Диаграмма состояний	16
2.5 Диаграммы классов.....	18
2.5.1 Слой домена	18
2.5.2 Реализации и инфраструктура	19
2.6 Диаграммы последовательности	20
2.6.1 Синхронная загрузка документа.....	20
2.6.2 Асинхронная индексация документа.....	21
2.6.3 Гибридный поиск.....	22
3 Результаты разработки программного продукта.....	25
3.1 Реализация требований к системе.....	25
3.1.1 Языки программирования и технологии.....	25
3.1.2 Корректность работы и обработка ошибок	25
3.1.3 Архитектура go-сервисов	26
3.1.4 Асинхронная очередь задач	26
3.1.5 Кэширование сессий и метаданных	28
3.1.6 Безопасность.....	28
3.1.7 Веб-интерфейс и пользовательский опыт	29
3.2 Функциональное тестирование программного продукта	30

3.2.1	Модульное тестирование	30
3.2.2	Тестирование пользовательских сценариев.....	31
3.3	Инструкция по эксплуатации	35
3.4	Итоги разработки	37
	Заключение	38
	Список используемых источников	40
	Приложения	42
	Приложение А — Исходный код Go-микросервисов.....	43
	Приложение Б — Исходный код Python-микросервисов.....	51
	Приложение В — Конфигурации Docker для развёртывания.....	55
	Приложение Г — Скриншоты веб-интерфейса системы	59
	Приложение Д — SQL-миграции и запросы к базе данных.....	66
	Приложение Е — Ссылка на репозиторий проекта	69

ВВЕДЕНИЕ

Объёмы научной, технической и корпоративной документации в формате PDF постоянно увеличиваются, что обуславливает необходимость в эффективных инструментах поиска. Простого полнотекстового сопоставления часто оказывается недостаточно — требуются интеллектуальные системы, способные понимать смысл запросов. Комбинирование традиционных методов индексации с семантическим анализом на основе нейросетевых эмбеддингов позволяет находить нужную информацию даже при отсутствии точных ключевых слов из документа в поисковом запросе.

Разработка подобного приложения с использованием микросервисной архитектуры, асинхронной обработки данных и современных подходов к хранению информации позволяет получить практический опыт в области промышленного программирования и искусственного интеллекта.

Цель работы: разработка веб-приложения на языке Go для интеллектуального поиска по PDF-документам, поддерживающего полнотекстовый и семантический режимы поиска, с распределённой архитектурой и асинхронной индексацией документов.

Задачи работы:

- Описание концепции системы интеллектуального поиска документов;
- Анализ существующих решений (полнотекстовые движки, векторные базы данных);
- Проектирование микросервисной архитектуры (Go-сервер, Python-эмбеддер, PostgreSQL + pgvector, Redis, MinIO);
- Реализация аутентификации с JWT и кэшированием сессий;
- Разработка асинхронного индексатора на базе очереди задач Async;
- Интеграция объектного хранилища MinIO для безопасного хранения файлов;
- Создание веб-интерфейса с использованием Flask и htmx;

- Проведение тестирования (unit, интеграционные тесты);
- Составление отчёта по работе и подготовка презентации.

Объектом исследования выступает процесс разработки интеллектуальных систем поиска по неструктурированным текстовым документам.

Предметом исследования выступают методы индексации, ранжирования и семантического поиска, реализованные на стеке Go + Python с применением микросервисной архитектуры, очередей задач и объектного хранения.

Методы исследования: анализ предметной области и существующих решений, синтез архитектурных компонентов, сравнение алгоритмов поиска (триграммы и векторные представления), моделирование процессов загрузки и фоновой обработки документов, а также экспериментальное тестирование производительности.

Информационная база исследования основана на открытых источниках — документации к PostgreSQL pgvector, sentence-transformers, Async и MinIO. Также были использованы материалы курса «Технологии промышленного программирования» РТУ МИРЭА и практические руководства по разработке на Go и Python.

В настоящем отчёте описываются функциональные требования к системе, её архитектура и детали реализации ключевых компонентов — загрузка файлов, извлечение текста, генерация эмбеддингов, а также полнотекстовый и семантический поиск. Кроме того, представлены результаты тестирования и инструкция по развёртыванию системы с помощью Docker Compose.

1 ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ

Системы поиска информации (Information Retrieval, IR) предназначены для нахождения релевантных документов по пользовательскому запросу. В зависимости от способа индексации и сопоставления различают классический полнотекстовый поиск (основанный на лексическом совпадении слов) и семантический (векторный) поиск (использующий нейросетевые эмбединги для улавливания смыслового сходства). Кроме того, системы могут быть классифицированы по следующим признакам:

- тип индексируемых данных (структурированные, полуструктурированные, неструктурированные);
- способ хранения индекса (на диске, в памяти, гибридный);
- архитектура (монолитная, микросервисная, распределённая);
- язык реализации и экосистема;
- стоимость владения и лицензирования;
- сложность развёртывания и администрирования;
- поддержка морфологии русского языка.

Для задачи поиска по PDF-документам в академической и корпоративной среде существует ряд открытых и коммерческих решений. Выбор подходящей платформы требует комплексного анализа, учитывающего не только функциональные возможности, но и экономические факторы, а также специфические требования к обработке русскоязычных текстов.

1.1 Традиционные полнотекстовые движки

Apache Lucene / Elasticsearch — одна из самых популярных библиотек полнотекстового поиска на Java, также доступная через REST-интерфейс в Elasticsearch. Lucene строит обратный индекс (inverted index) с использованием алгоритмов стемминга, фильтрации стоп-слов и ранжирования по TF-IDF / BM25. Для поддержки нечёткого поиска применяется расстояние Левенштейна

и n-граммы. Elasticsearch также предоставляет возможности для работы с PDF, извлекая текст через Apache Tika. Однако классический Lucene не поддерживает семантическое (векторное) сопоставление без внешних плагинов, хотя в последних версиях добавлена экспериментальная поддержка плотных векторов [1].

С точки зрения экономических факторов, Elasticsearch распространяется по лицензии Apache 2.0 и является бесплатным для базового использования. Однако коммерческая поддержка (Elasticsearch Service) и расширенные функции безопасности требуют подписки [2]. Сложность развёртывания оценивается как высокая: требуется настройка кластера, управление шардированием, мониторинг JVM и оптимизация индексов [3].

Важным аспектом для русскоязычных документов является поддержка морфологии. Elasticsearch включает встроенный анализатор для русского языка (Russian analyzer), который использует стемминг Портера и списки стоп-слов. Однако качество лемматизации для русского языка уступает специализированным решениям вроде rymorphy2 или Natasha [4]. Это может приводить к потере точности при поиске слов с богатой морфологией (например, «документ» не найдёт «документа», «документов» при использовании базового стемминга).

Недостатком является высокая сложность настройки и потребность в ресурсах для работы с большими индексами.

PostgreSQL с расширением pg_trgm — встроенное решение для нечёткого поиска по триграммам. Позволяет использовать оператор % и функцию similarity(), могут быть построены GIN-индексы для ускорения запросов. Этот подход эффективен для небольших и средних коллекций документов (до нескольких сотен тысяч), но полнотекстовый поиск без учёта морфологии (особенно для русского языка) даёт много ложных срабатываний [5].

PostgreSQL является полностью бесплатным решением с открытым исходным кодом (лицензия PostgreSQL). Развёртывание значительно проще,

чем Elasticsearch: достаточно установки пакета и подключения необходимых расширений. Однако масштабирование на несколько узлов требует ручной настройки репликации и шардирования.

Поддержка русской морфологии в `pg_trgm` ограничена: расширение работает на уровне символьных триграмм без учёта лингвистических особенностей языка. Это означает, что поиск слова «документ» не найдёт форму «документа» без использования дополнительных словарей или расширений вроде `hunspell`. Для улучшения качества поиска по русскоязычным текстам рекомендуется использовать полнотекстовый поиск PostgreSQL (`tsvector`) со специализированными конфигурациями или интегрировать внешние морфологические анализаторы [4].

1.2 Системы семантического (векторного) поиска

С развитием глубокого обучения появились модели, преобразующие текст в плотные векторные представления (`embedding`), сохраняющие семантическую близость. Для хранения и поиска таких векторов используются специализированные векторные базы данных:

- `pgvector` — расширение для PostgreSQL, добавляющее тип данных `vector` и операторы косинусного, евклидова и скалярного расстояния. Позволяет интегрировать семантический поиск в уже существующую реляционную схему, поддерживает индексы IVFFLAT и HNSW. Идеально подходит для прототипирования и небольших проектов [6]. Развёртывание минимально сложное (установка расширения), стоимость — бесплатно;

- FAISS (Facebook AI Similarity Search) — высокопроизводительная библиотека для поиска ближайших соседей, написанная на C++ с Python-обёрткой. Используется в крупных системах с миллиардами векторов, но требует отдельного сервиса и не поддерживает транзакционность. Это бесплатное решение (лицензия MIT), но его интеграция в production-систему требует значительных усилий [7];

– Qdrant / Weaviate / Milvus — полнофункциональные векторные базы данных, предоставляющие REST/gRPC API, фильтрацию по метаданным и распределённую архитектуру. Они предлагают готовые коннекторы для популярных моделей эмбедингов (sentence-transformers, OpenAI). Развёртывание и поддержка сложнее, чем pgvector: требуется настройка кластера, мониторинг и резервное копирование. Qdrant предлагает облачную версию с оплатой по использованию, что может быть экономически выгодно для переменной нагрузки [8].

Семантический поиск в целом не зависит от морфологии языка напрямую, поскольку векторные представления кодируют смысл текста целиком. Однако качество эмбедингов для русского языка может отличаться от английского в зависимости от используемой модели. Модели семейства sentence-transformers (например, LaBSE, paraphrase-multilingual) показывают хорошие результаты для русского языка [9].

1.3 Интегрированные платформы для поиска по документам

Некоторые коммерческие и открытые системы объединяют полнотекстовый и семантический подходы:

– Amazon Kendra это управляемый сервис от AWS, использующий глубокое обучение для поиска по неструктурированному тексту. Поддерживает загрузку PDF, HTML, Word и обеспечивает ответы с цитатами. Стоимость начинается от \$0.70 за час использования индекса плюс плата за запросы [10]. Это делает Kendra экономически неэффективной для небольших проектов и академических исследований. Развёртывание максимально простое (fully managed), но требует привязки к инфраструктуре AWS;

– Elasticsearch + Eland это плагин, позволяющий загружать модели PyTorch и выполнять векторный поиск поверх обратного индекса. В комбинации с ingest-конвейерами можно извлекать текст из PDF и получать

эмбединги. Однако настройка требует глубоких знаний в области машинного обучения и DevOps;

– Haystack (deepset) это фреймворк на Python для построения поисково-вопросно-ответных систем. Предоставляет готовые компоненты для индексации документов, конвертеры (PDF, DOCX), интеграцию с векторными БД (FAISS, Qdrant) и моделями эмбедингов. Позволяет быстро создать прототип, но написан на Python, что может быть проблемой для высоконагруженных систем. Развёртывание средней сложности: доступен Docker Compose для быстрого старта, но production-конфигурация требует тонкой настройки компонентов [11].

1.4 Анализ существующих решений

Сравнение ключевых характеристик рассматриваемых решений представлено в таблице 1.1. В сравнении учитываются не только функциональные возможности, но и экономические факторы, сложность развёртывания, а также специфические требования к обработке русскоязычных текстов.

Таблица 1.1 – Сравнение существующих решений

Решение	Полнотекстный	Семантический	PDF	Стоимость	Русская морфология
Elasticsearch	+++	+	++	Платно	++
PostgreSQL (pg_trgm)	++	++	–	Бесплатно	+
Qdrant	–	+++	–	Облако	+++
FAISS	–	+++	–	Бесплатно	+++
Amazon Kendra	+++	+++	+++	Дорого	+++
Haystack	++	++	++	Бесплатно	++
Разраб. система	++	++	+	Бесплатно	++

1.5 Интерпретация результатов сравнения

Проведённый анализ выявляет несколько ключевых закономерностей при выборе платформы для поиска по документам:

Стоимость и лицензирование. Открытые решения (PostgreSQL, Elasticsearch базовая версия, Haystack) позволяют избежать лицензионных отчислений, но требуют инвестиций в инфраструктуру и персонал. Amazon Kendra предлагает managed-решение, но при стоимости от \$0.70 в час минимальный годовой бюджет составляет несколько тысяч долларов даже для небольших проектов.

Сложность развёртывания. Существует компромисс между простотой начальной настройки и гибкостью масштабирования. SaaS-решения (Kendra) имеют минимальный порог входа, но ограничивают контроль над инфраструктурой. Elasticsearch и FAISS предоставляют максимальную гибкость, но требуют специализированных знаний в области DevOps и машинного обучения.

Поддержка русской морфологии. Традиционные полнотекстовые движки (Elasticsearch, PostgreSQL) обеспечивают базовый стемминг, но часто теряют точность при обработке сложных морфологических форм русского языка. Векторный поиск (Qdrant, FAISS, pgvector) теоретически не зависит от морфологии, но качество результатов определяется используемой моделью эмбедингов. Мультиязычные модели (LaBSE, paraphrase-multilingual) показывают приемлемое качество для русского языка [9].

Таким образом, анализ существующих решений показывает, что коробочные решения либо монолитны и сложны в развёртывании (Elasticsearch), либо ориентированы на Python и не обеспечивают лёгковесной микросервисной архитектуры на Go (Haystack), либо являются проприетарными и платными (Amazon Kendra). Использование связки PostgreSQL + pgvector + Go-сервера с асинхронной обработкой через Redis (Asynq) [12, 13] и отдельным Python-сервисом для эмбедингов позволяет достичь оптимального баланса между функциональностью, производительностью и простотой реализации, что и определило выбор архитектуры в данном курсовом проекте.

2 ОПИСАНИЕ ПРОГРАММИРУЕМОЙ СИСТЕМЫ

Разрабатываемая система представляет собой интеллектуальную платформу для полнотекстового и семантического поиска по PDF-документам. В данной главе описываются функциональные и нефункциональные требования к системе.

Для детального представления поведения системы в различных режимах, её статической структуры и динамики взаимодействия контейнеров разработаны диаграммы в нотации UML 2.5.

2.1 Функциональные требования

Система предоставляет пользователям возможность регистрации и входа с использованием JWT-токенов, обеспечивающих безопасный доступ к защищённым ресурсам.

При загрузке PDF-файлов до 50 МБ пользователь указывает метаданные: название документа, авторов, год публикации и категорию. Загруженные файлы сохраняются в объектном хранилище MinIO [14], где им присваиваются уникальные идентификаторы для корректного размещения и доступа.

После загрузки каждый документ автоматически проходит фоновую обработку, включающую извлечение текста, разбиение на смысловые чанки и генерацию векторных эмбеддингов через специализированный Python-сервис с последующим сохранением результатов в базе данных.

Поиск в системе реализован по двум направлениям: полнотекстовый поиск по названию и содержанию документов с использованием триграммного сравнения для нахождения документов при неточном совпадении запроса, а также векторный (семантический) поиск на основе косинусного расстояния между эмбеддингами поискового запроса и документов.

В системе реализована строгая изоляция данных: пользователь видит только собственные загруженные документы, доступ к документам других пользователей полностью блокируется на уровне архитектуры.

Для просмотра PDF-документов доступен встроенный просмотрщик на базе PDF.js, работающий через временные presigned-ссылки из MinIO. Удаление документов производится комплексно — с очисткой файла из объектного хранилища и соответствующей записи из базы данных.

2.2 Нефункциональные требования

Система обеспечивает время ответа на поисковый запрос не более 1 секунды для проиндексированных документов и поддерживает работу до 100 одновременных пользователей с сохранением приемлемой производительности.

Для упрощения развёртывания система реализована с использованием Docker Compose [15], который контейнеризирует все компоненты: Go-сервер, Python-сервис эмбедингов, PostgreSQL, Redis, MinIO, индексер обработки задач и Flask BFF (Backend For Frontend).

Для повышения производительности применяется кэширование: сессии пользователей хранятся в Redis с TTL 1 час, метаданные документов — с TTL 5 минут, что снижает нагрузку на базу данных. Обработка документов выполняется асинхронно через очередь задач Asynq на базе Redis с автоматическими повторными попытками при возникновении ошибок.

2.3 Архитектура системы

Для реализации описанных требований система построена по архитектуре микросервисов с чётким разделением ответственности между контейнерами и применением принципов Clean Architecture. Такая архитектура обеспечивает масштабируемость, отказоустойчивость и простоту развёртывания системы.

2.3.1 Основные компоненты системы

2.3.1.1 API-шлюз (Go)

Реализует архитектуру с тремя слоями: доменным (сущности предметной области и интерфейсы репозиториев), прикладным (сценарии использования и бизнес-логика) и инфраструктурным (внешние зависимости вроде базы данных, хранилища и очереди задач). Шлюз предоставляет REST API для работы с документами, аутентификации и поиска.

2.3.1.2 WEBUI Frontend (Flask)

построен по паттерну Backend-For-Frontend. Он использует серверный рендеринг с htmx для динамических обновлений [16], управление сессиями на базе Redis с JWT-токенами и адаптивный дизайн. Frontend обеспечивает интерфейс для загрузки, поиска и просмотра документов.

2.3.1.3 Микросервис эмбедингов (FastAPI)

Микросервис эмбедингов — это специализированный Python-сервис для генерации векторных эмбедингов текста с помощью трансформерных моделей, семантического ранжирования результатов поиска и извлечения ответов на вопросы из документов.

2.3.1.4 Инфраструктурные компоненты

Инфраструктурные компоненты включают PostgreSQL с расширением pgvector (основная база данных с поддержкой векторного поиска), Redis для кэширования сессий и реализации очереди задач Async, MinIO в качестве объектного хранилища для PDF-файлов и Nginx как обратный прокси и балансировщик нагрузки.

2.3.2 Взаимодействие контейнеров

Взаимодействие контейнеров построено на синхронном взаимодействии через REST API (WEBUI <-> API-шлюз <-> Микросервис эмбедингов) и асинхронном через очереди сообщений (API-шлюз -> очередь Asynq -> индексер обработки документов). Все компоненты используют общие хранилища — PostgreSQL, Redis и MinIO.

2.3.3 Потоки данных

Основной сценарий обработки документа включает следующие шаги:

1. Пользователь загружает PDF через WEBUI интерфейс.
2. Flask BFF передаёт файл и метаданные в Go API.
3. Go API сохраняет метаданные в PostgreSQL и файл в MinIO.
4. Go API публикует задачу обработки в очередь Asynq (Redis).
5. Воркер извлекает задачу, скачивает PDF из MinIO.
6. Воркер извлекает текст, разбивает на чанки, отправляет в микросервис эмбедингов.
7. Эмбединги сохраняются в PostgreSQL (pgvector).
8. Пользователь может выполнять полнотекстовый и семантический поиск

Такая архитектура обеспечивает разделение ответственности, горизонтальное масштабирование контейнеров и отказоустойчивость за счёт асинхронной обработки.

2.3.4 Диаграмма контейнеров

Для визуализации взаимодействия контейнеров системы применена диаграмма контейнеров в нотации C4. На рисунке 2.1 показано взаимодействие всех компонентов системы.

Диаграмма контейнеров системы "Document Archive System"

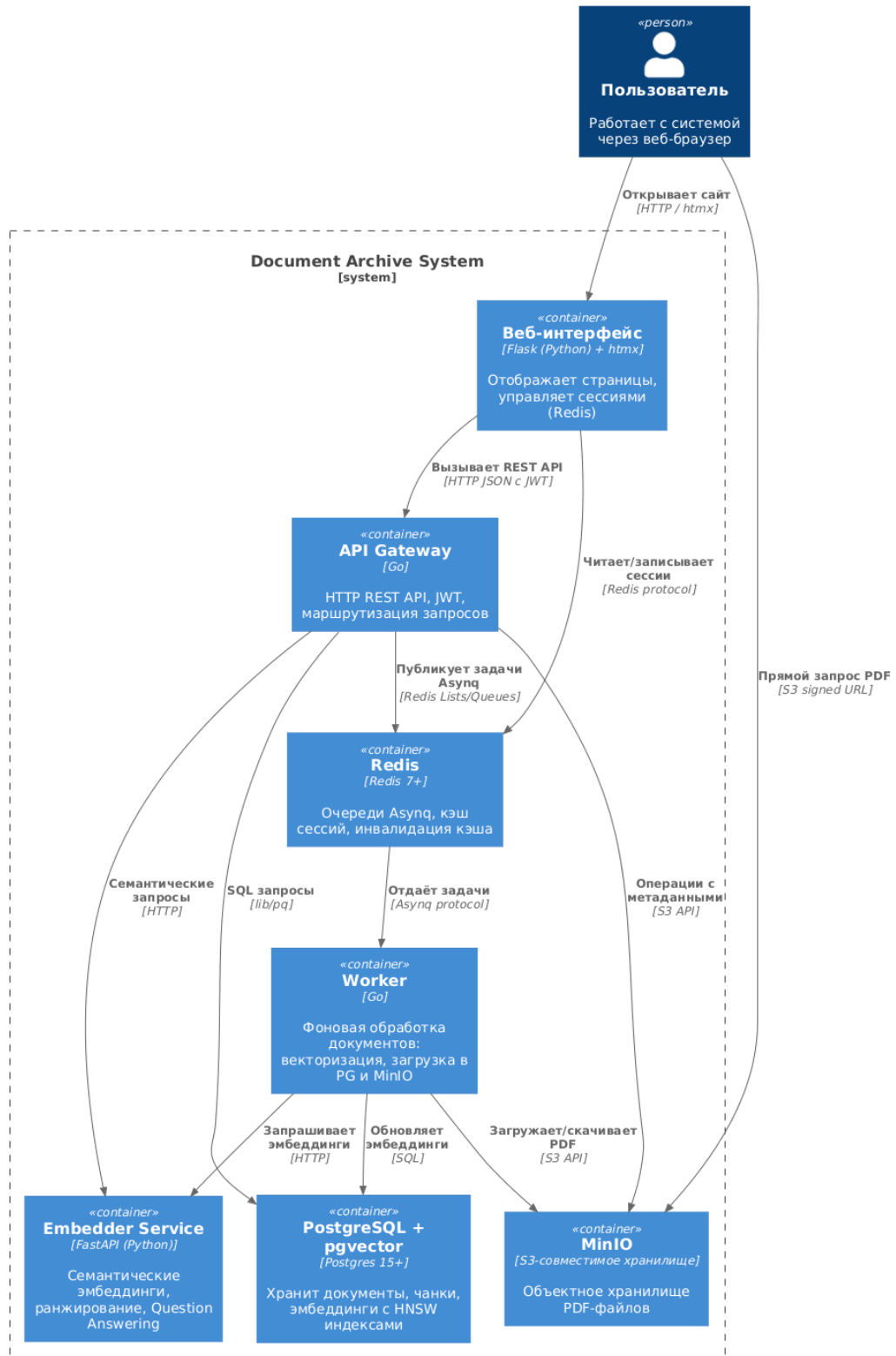


Рисунок 2.1 – Диаграмма контейнеров

2.4 Диаграмма состояний

Диаграмма состояний (рисунок 2.2) отображает жизненный цикл документа в системе — от загрузки до полной индексации или удаления.

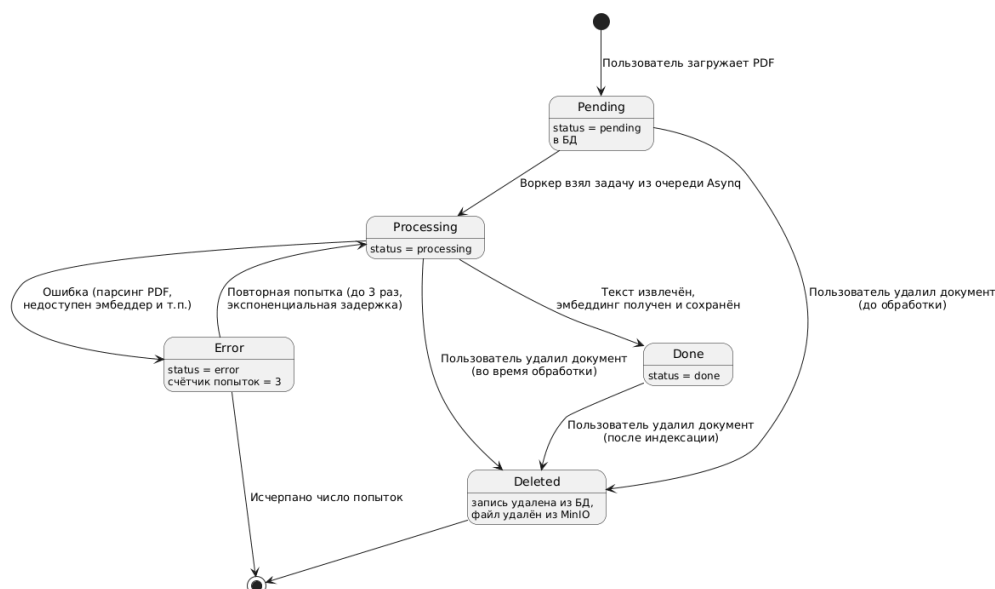


Рисунок 2.2 – Диаграмма состояний документа в системе

После загрузки PDF пользователем через веб-интерфейс создаётся запись в БД со статусом «pending» (ожидание обработки). Go-сервер публикует сообщение в очередь Asynq (тема process:document). Воркер, получив задачу, переводит документ в состояние «processing» (извлечение текста, вызов Python-сервиса для получения эмбединга, сохранение в MinIO — но файл уже сохранён ранее). При успешном завершении всех шагов статус меняется на «done». В случае неудачи (например, недоступность эмбедекера, ошибка парсинга PDF) задача повторяется до 3 раз с экспоненциальной задержкой; после исчерпания попыток статус становится «error».

Пользователь может в любой момент удалить документ, независимо от его состояния — при этом файл удаляется из MinIO, запись из БД, а также аннулируются связанные кэш-данные (Redis) и, если задача ещё в очереди, она отменяется. Документ также может быть удалён после успешной обработки — это действие переводит его в терминальное состояние «deleted» (запись фактически удаляется, поэтому на диаграмме это показано как переход в конечное состояние).

Диаграмма наглядно показывает гарантии обработки документов и механизмы восстановления системы после ошибок.

2.5 Диаграммы классов

Статическая структура Go-приложения представлена тремя взаимосвязанными диаграммами, которые вместе отражают архитектуру, построенную на принципах чистой архитектуры (Clean Architecture), применённых в версии 0.2.

2.5.1 Слой домена

На рисунке 2.3 показаны независимые сущности (модели), интерфейсы репозитория и интерфейсы сервисов, составляющие ядро приложения.

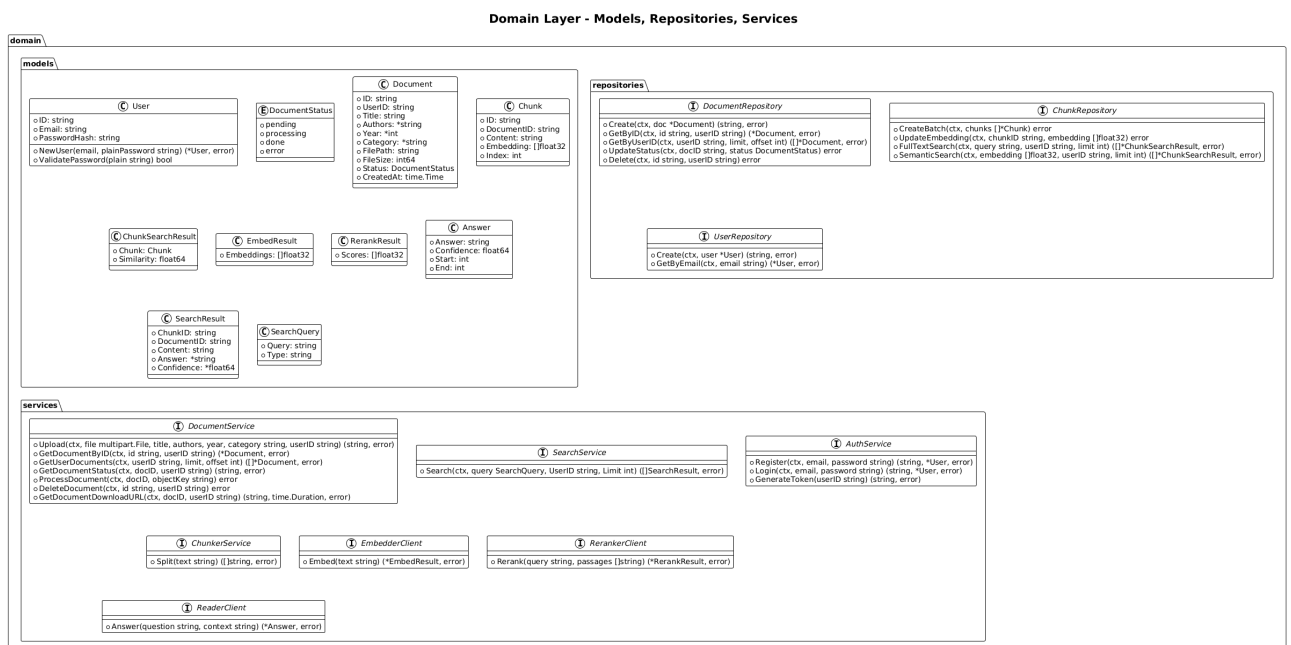


Рисунок 2.3 – Слой домена (модели, репозитории, сервисы)

В пакете доменов выделены:

- модели: User, Document, Chunk, а также вспомогательные DTO для поиска (SearchQuery, SearchResult) и операций выборки (Answer, RerankResult);
- интерфейсы репозитория: DocumentRepository, ChunkRepository, UserRepository. Они определяют методы доступа к данным, не привязываясь к конкретной СУБД;
- интерфейсы сервисов: DocumentService (управление документами), SearchService (гибридный поиск), AuthService (аутентификация), а также вспомогательные интерфейсы для интеграции с внешними компонентами:

ChunkerService (разбиение текста на чанки), EmbedderClient (генерация эмбеддингов), RerankerClient (переранжирование), ReaderClient (экстракция ответа).

Этот слой не имеет зависимостей от фреймворков и инфраструктуры, что обеспечивает тестируемость и независимость от деталей реализации.

2.5.2 Реализации и инфраструктура

На рисунке 2.4 показаны конкретные реализации репозиторий, сервисов и инфраструктурные адаптеры.

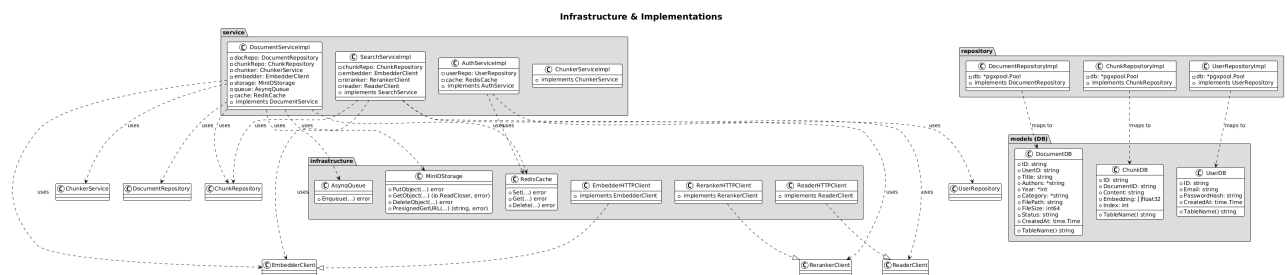


Рисунок 2.4 – Реализации репозиторий и инфраструктура

Слой репозиторий содержит реализации DocumentRepositoryImpl, ChunkRepositoryImpl, UserRepositoryImpl, работающие с PostgreSQL через pgxpool и отображающиеся на внутренние структуры DocumentDB, ChunkDB, UserDB (аннотированные для ORM). Слой сервисов предоставляет конкретные реализации интерфейсов домена:

- DocumentServiceImpl координирует загрузку файлов в MinIO, создание записей в БД, публикацию задач в очередь и кэширование через RedisCache.
- SearchServiceImpl объединяет полнотекстовый и семантический поиск, вызывая EmbedderClient, RerankerClient и ReaderClient.
- AuthServiceImpl отвечает за регистрацию/логин и генерацию JWT, сохраняя сессии в Redis.

Инфраструктурный пакет предоставляет конкретные реализации внешних зависимостей: MinIOStorage, RedisCache, AsyncQueue, а также HTTP-клиенты к Python-микросервисам (EmbedderHTTPClient,

RerankerHTTPClient, ReaderHTTPClient), которые реализуют соответствующие интерфейсы из домена.

Такая структура полностью соответствует принципу инверсии зависимостей (Dependency Inversion Principle): высокоуровневые модули (сервисы) зависят от абстракций (интерфейсов), а не от конкретных реализаций.

2.6 Диаграммы последовательности

В данном разделе представлены три диаграммы последовательности, которые иллюстрируют ключевые сценарии работы системы: загрузку PDF-документа с асинхронной индексацией (рисунки 2.5 и 2.6) и выполнение поискового запроса с возможностью переранжирования и генерации ответа (рисунок 2.7).

2.6.1 Синхронная загрузка документа

На рисунке 2.5 показан поток запроса при отправке PDF-файла и метаданных через браузер с участием промежуточного слоя Flask BFF (Backend For Frontend).

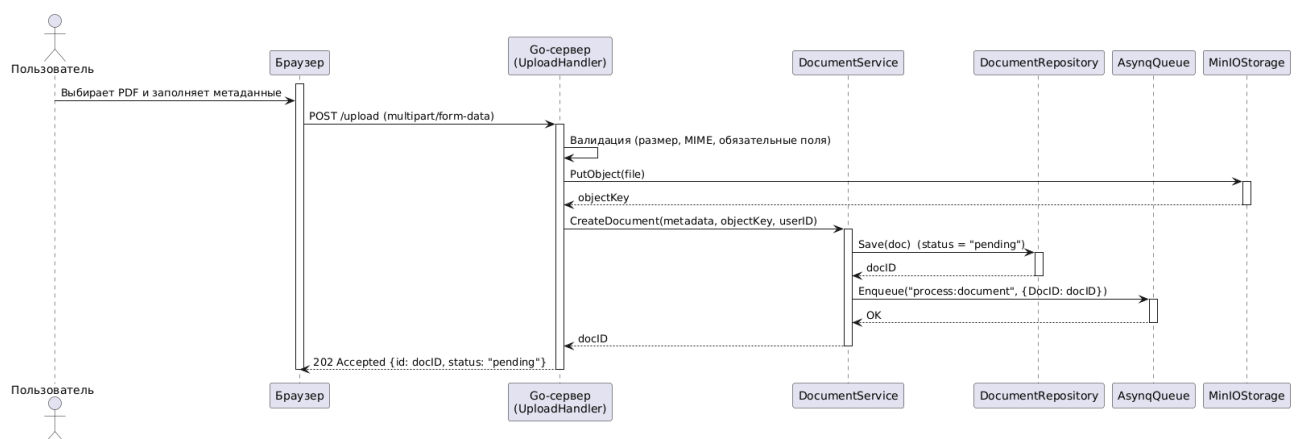


Рисунок 2.5 – Загрузка PDF (синхронная часть)

Пользователь через браузер выбирает PDF и заполняет метаданные (название, авторы, год, категория). Браузер отправляет POST-запрос «/upload» с типом multipart/form-data на Flask BFF. Flask выполняет первичное логирование

и валидацию параметров, затем пробрасывает запрос к Go-серверу по пути «/upload» вместе с заголовком Authorization. Go-обработчик UploadHandler проверяет размер файла, MIME-тип и обязательные поля, после чего сохраняет файл в MinIO через вызов PutObject. MinIO возвращает уникальный ключ объекта (objectKey). Далее вызывается метод CreateDocument сервиса DocumentService, который сохраняет запись в БД со статусом pending и публикует задачу process:document в очередь Asynq (Redis).

После успешного выполнения этих операций хендлер возвращает клиенту HTTP-статус 202 Accepted с JSON-телом, содержащим идентификатор документа и статус pending. Flask BFF транслирует этот ответ браузеру. Таким образом, пользователь получает мгновенный отклик, а ресурсоёмкая обработка документа выполняется асинхронно.

2.6.2 Асинхронная индексация документа

На рисунке 2.6 детализируется работа фонового индексера, который обрабатывает задание из очереди Asynq.

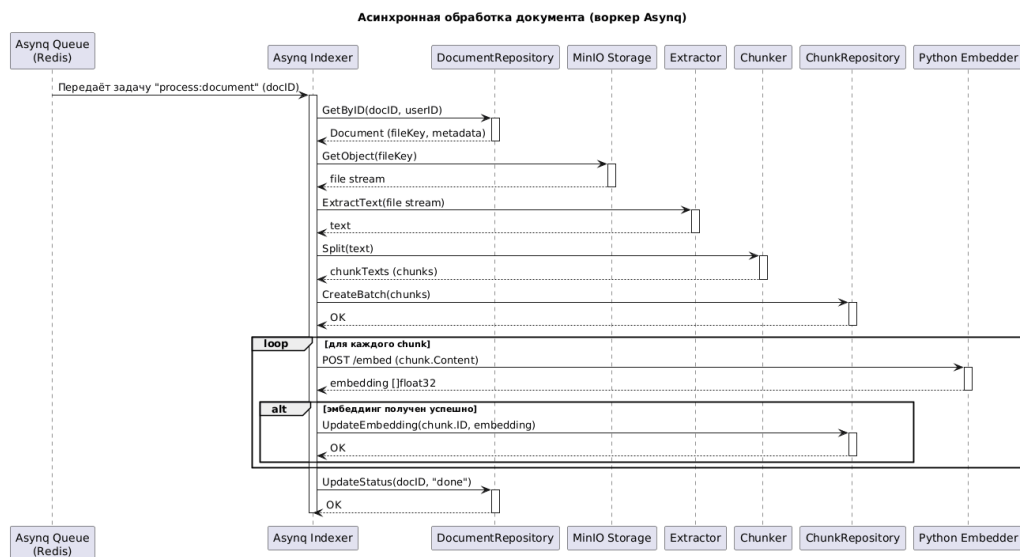


Рисунок 2.6 – Асинхронная обработка документа

Воркер Asynq (в коде — Asynq Indexer) извлекает из очереди задачу вида process:document, содержащую docID. Он запрашивает запись документа через DocumentRepository.GetByID, загружает файл из MinIO (GetObject), извлекает

текст с помощью Extractor (Go-библиотека github.com/AbdullahAlzariqi/pdf с фолбэком на утилиту pdftotext) и разбивает его на чанки через Chunker.Split. Для каждого чанка сначала вызывается CreateBatch (сохраняет чанки без эмбедингов), а затем в цикле для каждого чанка отправляется POST-запрос к сервису Python Embedder («/embed») для получения векторного представления. После успешного получения эмбединга обновляется запись чанка через UpdateEmbedding. По завершении всех чанков статус документа в БД меняется на done.

В случае ошибки на любом этапе задача будет автоматически повторена (согласно настройкам Asynq). Диаграмма наглядно демонстрирует, как система обеспечивает фоновую индексацию без блокировки основного потока обработки запросов пользователя.

2.6.3 Гибридный поиск

На рисунке 2.7 изображён процесс обработки поискового запроса, включающий текстовый или векторный поиск, опциональное переранжирование и чтение (генерацию ответа).

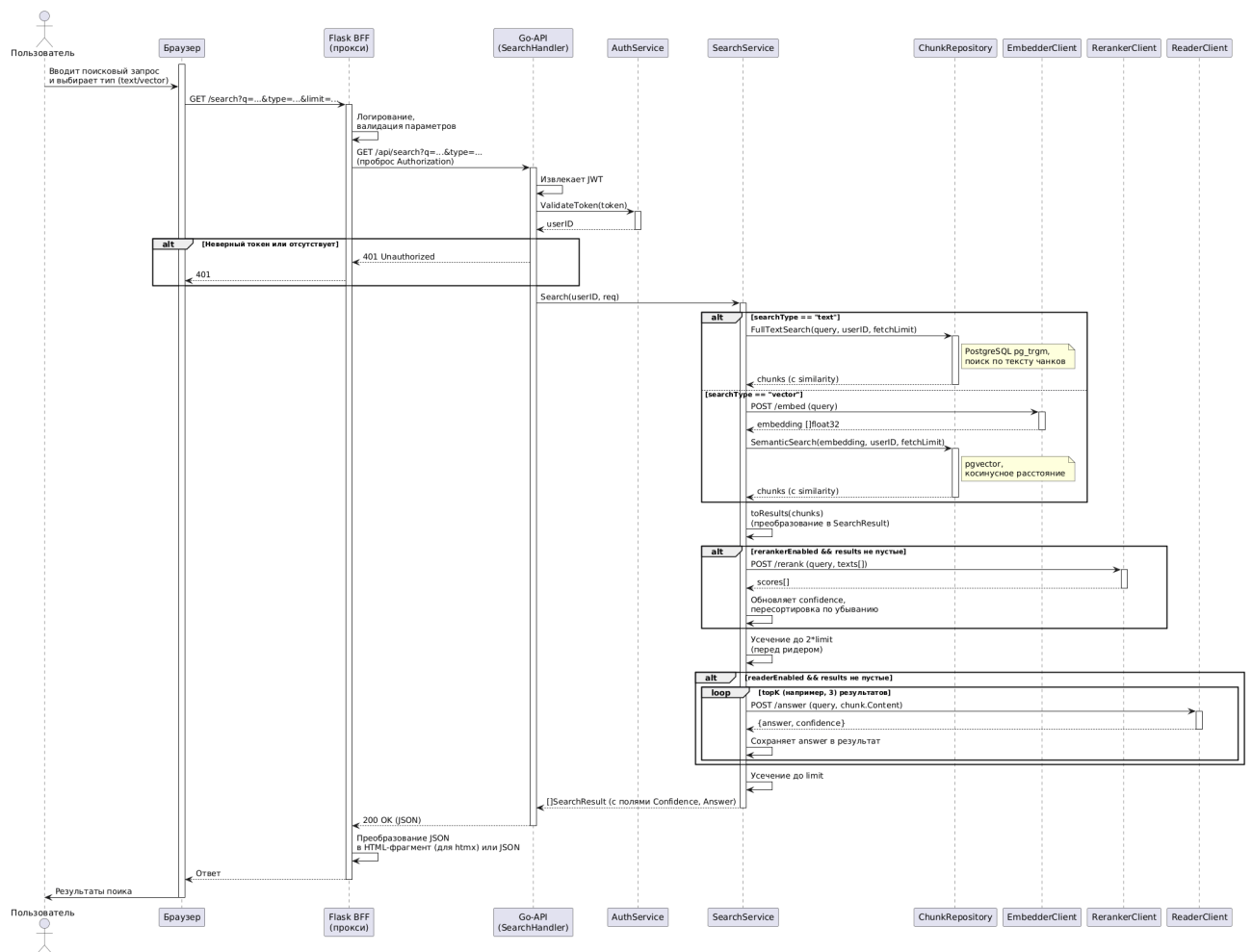


Рисунок 2.7 – Поиск с ранжированием и ответом

Пользователь вводит поисковую фразу и выбирает тип поиска (text или vector). Браузер отправляет GET-запрос на Flask BFF с параметрами q, type, limit. Flask после валидации передаёт запрос в Go-обработчик SearchHandler, который извлекает JWT из заголовка и вызывает AuthService.ValidateToken для получения идентификатора пользователя (при неверном токене возвращается 401). Далее вызывается метод Search сервиса SearchService. В зависимости от типа поиска выполняется либо полнотекстовый (через ChunkRepository.FullTextSearch с использованием pg_trgm), либо семантический (предварительно получается эмбединг запроса через EmbedderClient, затем ChunkRepository.SemanticSearch через pgvector). Полученные чанки преобразуются в структуру SearchResult. Если включён ранжировщик (rerankerEnabled) и результаты не пусты, вызывается внешний сервис Reranker для пересортировки списка по релевантности. После

этого при наличии ридера (readerEnabled) для первых topK (например, 3) результатов отправляется запрос к Reader для извлечения точного ответа на вопрос (используется модель extractive QA). Ответы сохраняются в соответствующих полях. Финальный список обрезаются до заданного limit и возвращается пользователю в формате JSON (Flask может преобразовать его в HTML-фрагмент для технологий вроде htmx). Диаграмма показывает, как система поддерживает гибридный поиск с возможностью расширения до полноценной генерации с дополненной выборкой (RAG).

Три представленные диаграммы последовательности вместе с требованиями, диаграммой состояний и диаграммами классов дают полное представление о системе, спроектированной и реализованной в рамках курсовой работы.

3 РЕЗУЛЬТАТЫ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА

Эта глава содержит итоги реализации системы интеллектуального поиска по PDF-документам. Здесь описаны достигнутые требования, представлены скриншоты рабочего интерфейса, ключевые фрагменты кода, результаты функционального тестирования и подробная инструкция для пользователей системы.

3.1 Реализация требований к системе

3.1.1 Языки программирования и технологии

- **Go 1.24** — основные микросервисы (HTTP, бизнес-логика, работа с БД, очередью, MinIO, кэшем);
- **Python 3.11 + FastAPI + sentence-transformers** — микросервис генерации эмбеддингов;
- **PostgreSQL 15 + pgvector** — хранение метаданных и векторов.
- **Redis** — брокер Asynq и кэш;
- **MinIO** — объектное хранилище;
- **Docker Compose** — контейнеризация.

Весь исходный коды (Go-сервер, Python-микросервис, конфигурации Docker) приведен в приложениях А–В и репозитории из приложения Е.

3.1.2 Корректность работы и обработка ошибок

Система сохраняет работоспособность даже при частичных сбоях благодаря нескольким механизмам обеспечения отказоустойчивости.

Если Python-эмбеддер временно недоступен, индексер автоматически помещает задачу в очередь для повторных попыток (до 3 раз) с экспоненциальной задержкой 2, 4, 8 секунд.

При ошибке извлечения текста (например, из-за битого PDF-файла) документ получает статус error, и пользователь получает понятное сообщение в интерфейсе.

При удалении документа во время его обработки задача автоматически отменяется из очереди, а файл удаляется из MinIO, что полностью исключает гонки данных.

3.1.3 Архитектура go-сервисов

Структура Go-сервисов организована согласно принципам чистой архитектуры и представлена в листинге 3.1.

Листинг 3.1 – Структура проекта

```
server/
├── cmd/
│   ├── api/           # точка входа HTTP сервера
│   └── indexer/       # точка входа Asynq индексер
├── internal/
│   ├── transport/     # транспортный слой
│   │   ├── http/      # HTTP обработчики, middleware, роутер
│   │   └── queue/      # Asynq обработчики и роутер задач
│   ├── service/       # Бизнес логика (DocumentService, AuthService)
│   ├── repository/    # слой доступа к данным (PostgreSQL)
│   ├── domain/        # бизнес сущности и интерфейсы
│   ├── infrastructure/ # Реализации внешних зависимостей
│   ├── tasks/         # типы задач для очереди (payload, константы)
│   ├── config/        # загрузка конфигурации
│   └── migrations/    # SQL миграции обычно ( выносятся в корень)
├── go.mod
├── go.sum
└── README.md
```

3.1.4 Асинхронная очередь задач

Для асинхронной обработки документов была выбрана библиотека Asynq, работающая поверх Redis. В листинге 3.2 представлено описание типа задачи и функция её создания, а в листинге 3.3 — полная реализация обработчика индексера, который выполняет шестишаговый конвейер: скачивание PDF, извлечение текста, разбиение на чанки, генерацию эмбеддингов, сохранение в БД и инвалидацию кэша.

Листинг 3.2 – Определение задачи

```
const TypeProcessDocument = "process:document"

type ProcessDocumentPayload struct {
    DocumentID string `json:"document_id"`
    ObjectKey   string `json:"object_key"`
}

func NewProcessDocumentTask(
    docID, objectKey string, opts ...asynq.Option,
) (*asynq.Task, error) {
    payload := ProcessDocumentPayload{
        DocumentID: docID,
        ObjectKey:   objectKey,
    }
    data, err := json.Marshal(payload)
    if err != nil {
        return nil, err
    }
    return asynq.NewTask(TypeProcessDocument, data, opts...), nil
}
```

Непосредственная реализация обработчика находится в листинге 3.3. Метод `ProcessDocument` последовательно выполняет шесть шагов конвейера обработки: получение метаданных из БД, скачивание PDF из MinIO, извлечение текста, разбиение на чанки по 500 символов, получение и усреднение эмбедингов, сохранение результатов и инвалидацию кэша.

Листинг 3.3 – Обработчик индексера

```
func (s *service) ProcessDocument(ctx context.Context, docID, objectKey string) error {
    reader, err := s.fileStorage.Download(ctx, objectKey)
    defer reader.Close()
    tmpFile, err := os.CreateTemp("", "pdf-*.pdf")
    defer os.Remove(tmpFile.Name())
    defer tmpFile.Close()
    err := io.Copy(tmpFile, reader)
    if err != nil { return fmt.Errorf("copy to temp: %w", err) }
    text, err := parser.ExtractFromPDF(tmpFile.Name())
    chunkTexts, err := s.chunker.Split(text)
    chunks := make([]*domain.Chunk, 0, len(chunkTexts))
    for idx, ct := range chunkTexts {
        chunks = append(chunks, &domain.Chunk{ DocumentID: docID, Index: idx, Content: ct })
    }
    err := s.chunkRepo.CreateBatch(ctx, chunks)
    if err != nil { return fmt.Errorf("copy to temp: %w", err) }
    for _, chunk := range chunks {
        embResp, err := s.embedder.Embed(chunk.Content)
        if err != nil { continue }
        err := s.chunkRepo.UpdateEmbedding(ctx, chunk.ID, embResp.Embeddings)
        if err != nil { continue }
    }
    return nil
}
```

Воркер запускается как отдельный сервис в Docker (контейнер indexer). Повторные попытки настраиваются через `asynq.RetryDelayFunc`.

3.1.5 Кэширование сессий и метаданных

Чтобы снизить нагрузку на PostgreSQL, сессии пользователей и списки документов кэшируются в Redis. В листинге 3.4 представлены методы записи и чтения сессии по токену, а в листинге 3.5 — механизм инвалидации кэша при удалении документов, обеспечивающий согласованность данных.

Листинг 3.4 – Кэш сессий (infrastructure/redis_cache.go)

```
func (c *RedisCache) SetSession(ctx context.Context, token string, userID int, ttl time.Duration) error {
    key := fmt.Sprintf("session:%s", token)
    return c.rdb.Set(ctx, key, userID, ttl).Err()
}

func (c *RedisCache) GetUserIDBySession(ctx context.Context, token string) (int, error) {
    key := fmt.Sprintf("session:%s", token)
    val, err := c.rdb.Get(ctx, key).Int()
    if errors.Is(err, redis.Nil) {
        return 0, ErrNotFound
    }
    return val, err
}
```

Листинг 3.5 демонстрирует инвалидацию кэша в методе `DeleteDocument`: при удалении документа очищаются оба ключа — для конкретного документа и для списка документов пользователя, что предотвращает выдачу устаревших данных.

Листинг 3.5 – Инвалидация кэша после обновления документа

```
// В DocumentService после удаления:
func (s *DocumentService) DeleteDocument(...) error {
    // ... удаление из БД и MinIO
    _ = s.cache.Delete(ctx, fmt.Sprintf("doc:%d", docID))
    _ = s.cache.Delete(ctx, fmt.Sprintf("user_docs:%d", userID))
    return nil
}
```

3.1.6 Безопасность

Для аутентификации в системе реализован стандарт JWT. В листинге 3.6 представлен метод генерации токена с временем жизни 1 час, а в листинге 3.7

— механизм формирования временных presigned-ссылок MinIO, позволяющий безопасно просматривать PDF-документы непосредственно в браузере без раскрытия учётных данных хранилища.

Листинг 3.6 – Генерация JWT (service/auth_service.go)

```
func (s *AuthService) GenerateToken(userID int) (string, error) {
    claims := jwt.MapClaims{
        "user_id": userID,
        "exp":     time.Now().Add(time.Hour).Unix(),
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString([]byte(s.cfg.JWTSecret))
}
```

Листинг 3.7 – Выдача presigned URL для просмотра PDF

```
func (s *MinIOStorage) GetPresignedURL(ctx context.Context, objectKey string, expiry time.Duration)
    reqParams := make(url.Values)
    reqParams.Set("response-content-disposition", fmt.Sprintf("inline; filename=\"%s\"", objectKey))
    return s.minioClient.PresignedGetObject(ctx, s.bucketName, objectKey, expiry, reqParams)
}
```

3.1.7 Веб-интерфейс и пользовательский опыт

Фронтенд реализован на Flask (шаблоны Jinja2) с htmx. Страницы:

- «/login» и «/register» — аутентификация;
- «/» — главная поисковая страница;
- «/upload» — загрузка документа;
- «/documents» — список документов пользователя (пагинация);
- «/documents/<id>» — просмотр документа (PDF.js + метаданные).

В листинге 3.8 приведён ключевой фрагмент HTML-шаблона страницы загрузки. Атрибуты hx-post и hx-trigger библиотеки htmx обеспечивают отправку файла без перезагрузки страницы, а результат помещается в блок.

Листинг 3.8 – Фрагмент шаблона *upload.html* (Jinja2 + htmx)

```
<div class="dropzone"
  hx-post="/upload"
  hx-trigger="drop"
  hx-target="#upload-status"
  hx-encoding="multipart/form-data">
  <input type="file" name="file" accept=".pdf" />
  <input type="text" name="title" placeholderНазвание=" *" required />
  <input type="text" name="authors" placeholderАвторы="" />
  <button type="submitЗагрузить"></button>
</div>
<div id="upload-status"></div>
```

3.2 Функциональное тестирование программного продукта

3.2.1 Модульное тестирование

Для обеспечения качества и надёжности Go-сервера системы была реализована комплексная стратегия модульного тестирования. Тесты покрывают ключевые компоненты архитектуры, включая доменные сущности, бизнес-логику и HTTP-обработчики. Результаты автоматического тестирования представлены на рисунке Г.14.

Доменные сущности достигнуто 100% покрытие тестами для структур User, Document и Chunk. Проверены валидация паролей с использованием bcrypt, корректность переходов статусов документов и граничные условия.

Покрытие тестами бизнес-логики составило:

- тестирование алгоритма разбиения текста на чанки фиксированного размера с поддержкой границ предложений — 90.1%;
- проверка работы семантического поиска, ранжирования результатов и фильтрации по пользователям — 96.3%;
- тестирование операций управления документами, включая обновление статусов и обработку ошибок — 42.5%.

Покрытие тестами HTTP-обработчиков:

- тестирование эндпоинтов для загрузки документов, поиска и управления документами — 72.6%;

– проверка JWT-аутентификации, авторизации и обработки ошибок — 41.5%;

– тестирование генерации и валидации JWT-токенов — 91.7%.

Общее покрытие кода по всему Go-проекту составляет 31.5%. Низкий процент объясняется отсутствием тестов для инфраструктурных слоёв (база данных, MinIO, Redis, внешние API), которые в рабочей системе покрываются интеграционными тестами и тестируются вручную в составе Docker-окружения.

3.2.2 Тестирование пользовательских сценариев

Функциональное тестирование системы интеллектуального поиска по PDF-документам проводилось вручную для всех сценариев использования, описанных в технических требованиях. Тестирование охватывало полный жизненный цикл работы пользователя с системой — от регистрации и аутентификации до загрузки документов, их обработки, поиска и управления.

Для каждого функционального модуля системы были разработаны и выполнены тестовые сценарии, проверяющие корректность работы в нормальных условиях, а также обработку пограничных случаев и ошибок. Все взаимодействия пользователя с системой документированы скриншотами, которые можно найти в приложении Г. Основные этапы тестирования и соответствующие скриншоты представлены в этом разделе.

3.2.2.1 Тестирование регистрации и аутентификации

При тестировании процессов регистрации и аутентификации основной фокус был направлен на проверку соответствия требованиям безопасности и удобства пользователя.

На рисунке Г.2 показана форма создания учётной записи. Тестирование включало проверку следующих аспектов: валидация email-адреса, требования к сложности пароля (минимум 6 символов), обработка дублирующихся

email-адресов, а также корректное хеширование паролей с использованием алгоритма bcrypt. Система предоставляет мгновенную обратную связь при вводе некорректных данных.

Форма аутентификации пользователя представлена на рисунке Г.3. Тестирование подтвердило несколько ключевых моментов: успешный вход с корректными учетными данными, защита от brute-force атак через ограничение попыток, установка HttpOnly cookie с JWT-токеном, валидация срока действия токена (1 час), а также корректное хранение сессий в Redis. После успешной аутентификации пользователь автоматически перенаправляется на главную страницу системы.

3.2.2.2 Тестирование загрузки и управления документами

При тестировании модуля загрузки документов основное внимание уделялось поддержке различных форматов файлов, обработке ошибок и удобству взаимодействия с пользователем.

Интерфейс для добавления PDF-документов в систему представлен на рисунке Г.4. В ходе тестирования проверялись следующие возможности: поддержка перетаскивания файлов (drag-and-drop), ограничение размера файла (до 50 МБ), валидация формата (только PDF), обязательность заполнения поля «Название», а также отображение индикатора прогресса загрузки. Система корректно обрабатывает прерывания загрузки и предоставляет пользователю понятные сообщения об ошибках.

Интерфейс управления документами пользователя представлен на рисунке Г.5. Тестирование подтвердило корректное отображение всех загруженных PDF-документов, отображение метаданных (название, авторы, год, категория), а также доступность кнопки удаления для каждого документа.

Детальный просмотр загруженного файла со встроенным просмотром представлен на рисунке Г.6.

3.2.2.3 Фоновая обработка (логи индексера)

В листинге 3.9 представлен фрагмент реального журнала индексера при обработке документа: последовательно фиксируются все шаги конвейера обработки — от скачивания PDF из MinIO до сохранения усреднённого вектора в pgvector и смены статуса документа на done.

Листинг 3.9 – Лог обработки документа

```
2025-04-26T10:15:23Z [INFO] Starting process document task id=42
2025-04-26T10:15:24Z [INFO] Downloaded from MinIO: articles/42.pdf (size 1.2 MB)
2025-04-26T10:15:25Z [INFO] Extracted text length: 12430 chars
2025-04-26T10:15:26Z [INFO] Calling embedder for 5 chunks...
2025-04-26T10:15:28Z [INFO] Averaged embedding (384 dim) saved to pgvector
2025-04-26T10:15:28Z [INFO] Document 42 status updated to 'done'
```

3.2.2.4 Тестирование поисковых механизмов

Поисковые возможности системы подверглись тщательному тестированию для проверки точности, релевантности результатов и производительности при работе с различными типами запросов.

Результаты поиска по запросу «language is a?» представлены на рисунке Г.7. Тестирование подтвердило несколько ключевых аспектов: корректную работу триграммного поиска PostgreSQL, формирование релевантных сниппетов с подсветкой ключевых слов, эффективное ранжирование результатов по релевантности, а также поддержку сложных запросов с логическими операторами. В этом тесте система вернула 2 документа, содержащих фразу об языке программирования.

Результаты для запроса «language is a?» в светлой теме интерфейса представлены на рисунках Г.7 и Г.8. Тестирование показало способность системы находить документы по смыслу, даже без точного вхождения ключевых слов: в результатах присутствуют документы о языках программирования. Система корректно вычисляет и отображает косинусное сходство в процентах, обеспечивая прозрачность ранжирования для пользователя.

В листинге 3.10 представлен SQL-запрос семантического поиска с использованием оператора косинусного расстояния «<=>» расширения pgvector. Тестирование подтвердило корректную работу векторных операций, эффективную фильтрацию по user_id для обеспечения изоляции данных между пользователями, а также оптимальную производительность при работе с большими объемами векторных данных.

Листинг 3.10 – Пример SQL для семантического поиска

```
SELECT c.id, d.title, d.authors, d.year, d.category,  
       c.content, 1 - (c.embedding <=> $1) AS similarity  
FROM chunks c  
JOIN documents d ON c.document_id = d.id  
WHERE d.user_id = $2 AND c.embedding IS NOT NULL  
ORDER BY c.embedding <=> $1  
LIMIT $3;
```

3.2.2.5 Тестирование удаления документов и кэширования

При тестировании модуля удаления документов основное внимание уделялось безопасности, атомарности операций и согласованности данных между различными компонентами системы.

Механизм безопасного удаления документов представлен на рисунке Г.9. Тестирование подтвердило несколько важных аспектов: обязательное подтверждение пользователя перед удалением, атомарное выполнение операции (удаление записи из PostgreSQL, файла из MinIO, очистка кэша Redis), корректную инвалидацию связанных данных в кэше, а также асинхронное обновление интерфейса без перезагрузки страницы. Система эффективно предотвращает гонки данных при параллельных операциях.

3.2.2.6 Тестирование инфраструктуры и API

Инфраструктурные компоненты системы прошли тестирование на стабильность работы, отказоустойчивость и корректность интеграции между различными сервисами.

Корректная оркестрация всех сервисов системы представлена на рисунке Г.1. Тестирование подтвердило несколько ключевых моментов: успешный запуск и взаимодействие контейнеров, правильную настройку сетевых связей между компонентами, корректную обработку перезапусков сервисов, а также эффективную работу health-check механизмов.

Состояние объектного хранилища MinIO представлено на рисунке Г.10. Тестирование выявило важные возможности системы: корректное сохранение и извлечение PDF-файлов, работу presigned-ссылок с ограниченным временем действия, обработку ошибок при недоступности хранилища, а также соблюдение политик безопасности доступа.

Интерактивная документация всех REST API системы представлена на рисунках Г.11, Г.12, Г.13. Тестирование подтвердило несколько важных аспектов: корректность описания эндпоинтов, работу авторизации через JWT, валидацию входных параметров, обработку ошибок в стандартизированном формате, а также возможность тестирования API непосредственно из документации.

Все разработанные тестовые сценарии были успешно выполнены, и система продемонстрировала стабильную работу во всех предусмотренных сценариях использования.

3.3 Инструкция по эксплуатации

Для работы системы потребуются следующие компоненты: Python версии 3.11 или выше, установленные Docker и Docker Compose версии 2.0 или выше, 2 ГБ свободной оперативной памяти и около 15 ГБ дискового пространства.

Чтобы запустить систему, выполните следующие шаги. Сначала клонируйте репозиторий с исходным кодом. И перейдите в директорию проекта (Листинг 3.11).

Листинг 3.11 – Клонирование репозитория

```
git clone https://github.com/AndB0ndar/doc-archive.git  
cd doc-archive
```

Запустите приложение с помощью Docker Compose. Наконец, откройте браузер и перейдите по локального хоста (Листинг 3.12).

Листинг 3.12 – Развёртывание системы

```
docker-compose up -d  
http://localhost:5005
```

Для начала работы с системой выполните следующие действия. Нажмите кнопку «Зарегистрироваться», затем введите email и пароль (пароль должен содержать минимум 6 символов). После успешной регистрации войдите в систему с указанными учетными данными.

Чтобы добавить PDF-документы в систему, выполните следующие действия. Перейдите в раздел «Загрузить» в основном меню. Перетащите PDF-файл в выделенную область на странице или выберите файл через стандартный диалог. Заполните поле «Название» (обязательно), а также опциональные поля «Авторы», «Год» и «Категория» при необходимости. После подтверждения документ исчезает из списка, а соответствующий файл полностью удаляется из MinIO.

Чтобы завершить работу с системой, выполните следующие действия. Для выхода из учётной записи используйте пункт меню «Выйти». Чтобы остановить контейнеры и освободить системные ресурсы, выполните команду (Листинг 3.13).

Листинг 3.13 – Остановка контейнеров

```
docker-compose down
```

3.4 Итоги разработки

Разработанная система интеллектуального поиска по PDF-документам полностью соответствует всем функциональным и нефункциональным требованиям, изложенным в техническом задании.

В ходе разработки система успешно прошла все этапы тестирования, включая проверку аутентификации, загрузки документов, фоновой индексации через Asynq, полнотекстового и семантического поиска, удаления документов, работы с кэшированием и обеспечения безопасности. Веб-интерфейс системы интуитивно понятен, поддерживает переключение между тёмной и светлой темами, а также адаптивную вёрстку для различных устройств. Кодовая база организована по принципам чистой архитектуры, что обеспечивает простоту дальнейшего расширения функциональности (например, добавление OCR-обработки, поддержки других форматов документов и т.д.).

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы была спроектирована и реализована система интеллектуального поиска по PDF-документам, объединяющая классический полнотекстовый и семантический векторный поиск. На основе исследования существующих решений — Elasticsearch, pgvector, Qdrant и Amazon Kendra — была разработана оптимальная архитектура: Go-сервер для бизнес-логики, Python-микросервис для генерации эмбеддингов, PostgreSQL с расширением pgvector, Redis для очередей и кэширования, а также MinIO для хранения файлов.

На основе анализа требований были разработаны диаграммы состояний, классов и последовательности, которые наглядно демонстрируют жизненный цикл документа, статическую структуру и динамику взаимодействия компонентов. Реализованная система успешно решает поставленные задачи: поддерживает аутентификацию с JWT и хранением сессий в Redis, загрузку PDF через Drag&Drop, асинхронную индексацию через Asynq, полнотекстовый поиск на основе триграмм (pg_trgm) и семантический поиск с pgvector, кэширование метаданных и безопасное скачивание через presigned-ссылки MinIO.

Реализация этого проекта потребовала применения принципов чистой архитектуры и объектно-ориентированного программирования — инкапсуляции, внедрения зависимостей и программирования на интерфейсах. Работа с современными инструментами (Docker Compose, Redis, MinIO, Asynq, pgvector) позволила создать стабильную систему, устойчивую к сбоям благодаря автоматическим повторным попыткам обработки задач. Высокая производительность обеспечена кэшированием и асинхронной обработкой, а адаптивный веб-интерфейс с поддержкой тёмной темы и интерактивностью через htmx делает систему удобной для пользователей.

Наибольшую сложность представили задачи интеграции разнородных технологий — Go, Python, PostgreSQL, Redis и MinIO — в единую систему. Организация фоновой обработки с гарантией доставки через Async, разбиение текста на чанки и усреднение эмбедингов для больших документов, а также обеспечение безопасности (JWT, bcrypt, presigned URL, защита от CSRF) потребовали детального изучения каждой технологии. Система управления версиями Git позволила эффективно отслеживать изменения и поддерживать организованную кодовую базу.

Разработанная система открывает интересные направления для дальнейшего развития:

- добавление OCR для распознавания текста в сканированных PDF;
- поддержку других форматов документов (DOCX, EPUB);
- реализацию гибридного поиска (комбинация полнотекстового и семантического рангов);
- внедрение мониторинга (Prometheus + Grafana) и распределённой трассировки (OpenTelemetry);
- масштабирование индексов горизонтально для увеличения пропускной способности.

В ходе работы был получен ценный опыт в промышленной разработке на Go, понимании микросервисной архитектуры, работе с векторными базами данных и контейнеризации. Подтверждено, что объединение полнотекстового и семантического поиска создаёт более эффективную систему для работы с документами. Полученные навыки и понимание архитектурных принципов могут быть полезны при разработке высоконагруженных информационных систем и систем с элементами искусственного интеллекта.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. *Elasticsearch Documentation*. Vector search (kNN). — URL: <https://www.elastic.co/guide/en/elasticsearch/reference/current/knn.html> (дата обр. 15.04.2026).
2. *Elastic N.V.* Elasticsearch Licensing Guide. — URL: <https://www.elastic.co/pricing> (дата обр. 03.05.2026).
3. *Elasticsearch Documentation*. Elasticsearch: The Definitive Guide — Administration and Deployment. — URL: <https://www.elastic.co/guide/en/elasticsearch/guide/current/administration.html> (дата обр. 03.05.2026).
4. *Natasha Contributors*. Natasha: Natural Language Processing for Russian Language. — URL: <https://natasha.github.io/> (дата обр. 03.05.2026).
5. *PostgreSQL Documentation*. pg_trgm. — URL: <https://www.postgresql.org/docs/current/pgtrgm.html> (дата обр. 15.04.2026).
6. *pgvector Contributors*. pgvector GitHub repository. — URL: <https://github.com/pgvector/pgvector> (дата обр. 15.04.2026).
7. *Facebook Research*. FAISS: A library for efficient similarity search. — URL: <https://github.com/facebookresearch/faiss> (дата обр. 03.05.2026).
8. *Qdrant Technologies*. Qdrant Vector Database Documentation. — URL: <https://qdrant.tech/documentation/> (дата обр. 03.05.2026).
9. *SBERT.net*. Sentence-Transformers Documentation. — URL: <https://www.sbert.net> (дата обр. 26.04.2026).
10. *Amazon Web Services*. Amazon Kendra Pricing. — URL: <https://aws.amazon.com/kendra/pricing/> (дата обр. 03.05.2026).
11. *deepset GmbH*. Haystack Documentation — Installation and Deployment. — URL: <https://docs.haystack.deepset.ai/docs/installation> (дата обр. 03.05.2026).
12. *Redis*. Redis. — URL: <https://redis.io> (дата обр. 26.04.2026).
13. *Hibiken*. Async: Simple, reliable & efficient distributed task queue in Go. — URL: <https://github.com/hibiken/async> (дата обр. 26.04.2026).

14. *MinIO*. MinIO Quickstart Guide. — URL: <https://min.io/docs/minio/linux/index.html> (дата обр. 26.04.2026).
15. *Docker Documentation*. Docker Compose Overview. — URL: <https://docs.docker.com/compose> (дата обр. 26.04.2026).
16. *htmx.org*. htmx: high power tools for HTML. — URL: <https://htmx.org> (дата обр. 26.04.2026).

ПРИЛОЖЕНИЯ

Приложение А. Исходный код Go-микросервисов

Приложение Б. Исходный код Python-микросервисов

Приложение В. Конфигурации Docker для развёртывания

Приложение Г. Скриншоты веб-интерфейса системы

Приложение Д. SQL-миграции и запросы к базе данных

Приложение Е. Ссылка на репозиторий проекта

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД GO-МИКРОСЕРВИСОВ

Этот раздел содержит ключевые части Go-сервера, который обрабатывает запросы пользователей и координирует работу всей системы. Полный исходный код доступен в репозитории GitHub (Приложение Е).

1.1 Точка входа в API-Gateway приложение

Главный файл приложения выполняет две ключевые задачи: загрузку конфигурации из переменных окружения и запуск HTTP-сервера. Swagger-аннотации в начале файла автоматически генерируют OpenAPI-документацию, что позволяет разработчикам тестировать API через интерактивный интерфейс.

Листинг А.1 – cmd/api/main.go

```
// @title           PDF Search API
// @version          1.0
// @description      API для интеллектуального поиска по документам.
// @contact.name     arbon
// @license.name     MIT
// @license.url      https://opensource.org/licenses/MIT
// @host             localhost:8080
// @BasePath        /
// @securityDefinitions.apikey BearerAuth
// @in header
// @name Authorization
package main
import (
    "log/slog"
    "os"
    "github.com/AndB0ndar/doc-archive/internal/app"
    "github.com/AndB0ndar/doc-archive/internal/config"
)
func main() {
    cfg, err := config.Load()
    if err != nil {
        slog.Error("failed to load config", "error", err)
        os.Exit(1)
    }
    application := app.New(cfg)
    if err := application.RunAPI(); err != nil {
        slog.Error("API failed", "error", err)
        os.Exit(1)
    }
}
```

1.2 Домен и интерфейсы (internal/domain/document.go)

Доменный слой определяет центральные концепции системы — типы данных и интерфейсы, изолированные от инфраструктурных деталей. Этот подход позволяет менять базы данных или фреймворки, не затрагивая бизнес-логику приложения.

Перечисление `DocumentStatus` описывает жизненный цикл документа, а структура `Document` хранит все метаданные файла. Интерфейсы репозитория и сервиса задают контракты для работы с документами, не привязывая реализацию к конкретным технологиям.

Листинг A.2 – `internal/domain/document.go`

```
package domain
import ( "context" "mime/multipart" "time")
type DocumentStatus string
const (
    DocumentStatusPending    DocumentStatus = "pending"
    DocumentStatusProcessing DocumentStatus = "processing"
    DocumentStatusDone       DocumentStatus = "done"
    DocumentStatusError       DocumentStatus = "error"
)
type Document struct {
    ID          string
    UserID      string
    Title       string
    Authors     *string
    Year        *int
    Category    *string
    FilePath    string
    FileSize    int64
    Status      DocumentStatus
    CreatedAt   time.Time
}
type DocumentRepository interface {
    Create(ctx context.Context, doc *Document) (string, error)
    GetByID(ctx context.Context, id string, userID string) (*Document, error)
    GetByUserID(ctx context.Context, userID string, limit, offset int) ([]*Document, error)
    UpdateStatus(ctx context.Context, docID string, status DocumentStatus) error
    Delete(ctx context.Context, id string, userID string) error
}
type DocumentService interface {
    Upload(ctx context.Context, file multipart.File, title, authors, year, category string, userID string) error
    GetDocumentByID(ctx context.Context, id string, userID string) (*Document, error)
    GetUserDocuments(ctx context.Context, userID string, limit, offset int) ([]*Document, error)
    GetDocumentStatus(ctx context.Context, docID, userID string) (string, error)
    ProcessDocument(ctx context.Context, docID, objectKey string) error
    DeleteDocument(ctx context.Context, id string, userID string) error
    GetDocumentDownloadURL(ctx context.Context, docID, userID string) (string, time.Duration, error)
}
```

1.3 Сервис документов (internal/service/document_service.go)

Метод Upload обрабатывает загрузку PDF-файлов и координирует несколько систем. Он проверяет входные данные, загружает файл в MinIO, сохраняет метаданные в PostgreSQL и помещает задачу обработки в очередь Asynq.

Особенно важна механизм отката: если сохранение в базу данных не удаётся, файл автоматически удаляется из MinIO, предотвращая появление «осиротевших» файлов. Это гарантирует атомарность операции — либо все шаги выполняются успешно, либо ни одного.

Листинг A.3 – Фрагмент document_service.go

```
func (s *service) Upload(
    ctx context.Context,
    file multipart.File,
    title, authors, year, category string,
    userID string,
) (string, error) {
    if title == "" { return "", fmt.Errorf("title is required") }
    var authorsPtr *string
    if authors != "" { authorsPtr = &authors }
    var yearPtr *int
    if year != "" {
        var y int
        err := fmt.Sscanf(year, "%d", &y)
        if err == nil && y > 0 && y <= time.Now().Year()+1 { yearPtr = &y }
    }
    var categoryPtr *string
    if category != "" { categoryPtr = &category }
    objectKey := uuid.New().String() + ".pdf"
    fileBytes, err := io.ReadAll(file)
    if err != nil { return "", fmt.Errorf("read file: %w", err) }
    reader := bytes.NewReader(fileBytes)
    err = s.minio.Upload(ctx, objectKey, reader, int64(len(fileBytes)), "application/pdf")
    if err != nil { return "", fmt.Errorf("upload to minio: %w", err) }
    doc := &domain.Document{
        Title: title, Authors: authorsPtr, Year: yearPtr, Category: categoryPtr,
        FilePath: objectKey, FileSize: int64(len(fileBytes)),
        Status: domain.DocumentStatusPending,
        UserID: userID,
    }
    if id, err := s.docRepo.Create(ctx, doc); err != nil {
        _ = s.minio.Delete(ctx, objectKey) // remove file, if not save in DB
        return "", fmt.Errorf("save metadata: %w", err)
    }
    if err := s.enqueueProcessDocument(ctx, id, objectKey); err != nil {
        _ = s.docRepo.UpdateStatus(ctx, id, domain.DocumentStatusError)
    }
    return id, nil
}
```

1.4 Асинхронный индексатор (Asynq Worker)

Система использует распределённую очередь задач Asynq поверх Redis для фоновой обработки PDF-документов. Это позволяет отделить тяжёлые операции индексации (извлечение текста, генерация эмбедингов) от HTTP-запросов пользователей, обеспечивая быстрый отклик и устойчивость к сбоям.

1.4.1 Точка входа индексатора (cmd/indexer/main.go)

Индексатор представляет собой отдельный исполняемый файл, который запускается как фоновый воркер. Он загружает ту же конфигурацию, что и API-шлюз, но вызывает метод RunIndexer, отвечающий за обработку задач из очереди Asynq.

Листинг A.4 – cmd/indexer/main.go

```
package main
import (
    "log/slog"
    "os"
    "github.com/AndB0ndar/doc-archive/internal/app"
    "github.com/AndB0ndar/doc-archive/internal/config"
)
func main() {
    cfg, err := config.Load()
    if err != nil {
        slog.Error("failed to load config", "error", err)
        os.Exit(1)
    }
    application := app.New(cfg)
    if err := application.RunIndexer(); err != nil {
        slog.Error("Indexer failed", "error", err)
        os.Exit(1)
    }
}
```

1.4.2 Определение задач (internal/tasks/tasks.go)

Файл определяет типы задач, которые могут быть поставлены в очередь. Каждая задача имеет уникальный тип (TypeProcessDocument), сериализуемую полезную нагрузку и опциональные параметры (таймауты, приоритет, время отложенного выполнения).

Листинг A.5 – internal/tasks/tasks.go

```
package tasks
import (
    "encoding/json"
    "github.com/hibiken/asynq"
)
const TypeProcessDocument = "process:document"
type ProcessDocumentPayload struct {
    DocumentID string `json:"document_id"`
    ObjectKey   string `json:"object_key"`
}
func NewProcessDocumentTask(
    docID, objectKey string, opts ...asynq.Option,
) (*asynq.Task, error) {
    payload := ProcessDocumentPayload{
        DocumentID: docID,
        ObjectKey:  objectKey,
    }
    data, err := json.Marshal(payload)
    if err != nil {
        return nil, err
    }
    return asynq.NewTask(TypeProcessDocument, data, opts...), nil
}
```

1.4.3 Обработчик задачи индексации (internal/transport/queue/handlers/document.go)

Обработчик задачи извлекает полезную нагрузку из очереди, вызывает бизнес-логику обработки документа и обеспечивает логирование результатов. При возникновении ошибки задача автоматически перезапускается с экспоненциальной задержкой благодаря настройкам Asynq.

Листинг A.6 – *internal/transport/queue/handlers/document.go*

```
package handlers
import (
    "context"
    "encoding/json"
    "github.com/hibiken/asynq"
    "github.com/AndB0ndar/doc-archive/internal/domain"
    "github.com/AndB0ndar/doc-archive/internal/infrastructure/logger"
    "github.com/AndB0ndar/doc-archive/internal/tasks"
)
type DocumentHandler struct {
    docService domain.DocumentService
    logger      *logger.Logger
}
func NewDocumentHandler(
    docService domain.DocumentService,
    logger *logger.Logger,
) *DocumentHandler {
    return &DocumentHandler{ docService: docService, logger: logger, }
}
func (h *DocumentHandler) ProcessDocument(ctx context.Context, task *asynq.Task) error {
    var payload tasks.ProcessDocumentPayload
    if err := json.Unmarshal(task.Payload(), &payload); err != nil { return err }
    h.logger.Info(
        "processing document", "doc_id", payload.DocumentID, "object_key", payload.ObjectKey)
    err := h.docService.ProcessDocument(ctx, payload.DocumentID, payload.ObjectKey)
    if err != nil {
        h.logger.Error("processing failed", "doc_id", payload.DocumentID, "error", err)
        return err
    }
    h.logger.Info(
        "processing document completed", "doc_id", payload.DocumentID, "object_key", payload.ObjectKey)
    return nil
}
```

1.4.4 Сервер Asynq (internal/infrastructure/queue/server.go)

Сервер Asynq управляет пулом воркеров, которые обрабатывают задачи из очереди Redis. Конфигурация включает настройку параллелизма, приоритетов очередей и стратегии повторных попыток с экспоненциальной задержкой.

Листинг А.7 – *internal/infrastructure/queue/server.go*

```
package queue
import (
    "time"
    "github.com/hibiken/asynq"
)
type Server struct {
    asynq *asynq.Server
}
func NewServer(RedisAddr string, RedisDB int, Concurrency int) *Server {
    redisOpt := asynq.RedisClientOpt{Addr: RedisAddr, DB: RedisDB}
    srv := asynq.NewServer(redisOpt, asynq.Config{
        Concurrency: Concurrency,
        Queues:      map[string]int{"default": 1},
        RetryDelayFunc: func(n int, err error, task *asynq.Task) time.Duration {
            return time.Duration(1<<uint(n)) * time.Second
        },
    })
    return &Server{asynq: srv}
}
func (s *Server) Run(mux *asynq.ServeMux) error {
    return s.asynq.Run(mux)
}
func (s *Server) Stop() {
    s.asynq.Shutdown()
}
```

1.4.5 Маршрутизатор задач (*internal/transport/queue/router.go*)

Маршрутизатор связывает типы задач с соответствующими обработчиками. Эта простая, но важная компонента обеспечивает расширяемость системы: добавление новой задачи требует только регистрации нового обработчика в маршрутизаторе.

Листинг А.8 – *internal/transport/queue/router.go*

```
package queue
import (
    "github.com/hibiken/asynq"
    "github.com/AndB0ndar/doc-archive/internal/tasks"
    "github.com/AndB0ndar/doc-archive/internal/transport/queue/handlers"
)
func NewRouter(docHandler *handlers.DocumentHandler) *asynq.ServeMux {
    mux := asynq.NewServeMux()
    mux.HandleFunc(tasks.TypeProcessDocument, docHandler.ProcessDocument)
    return mux
}
```

1.4.6 Метод RunIndexer приложения (internal/app/app.go)

Метод RunIndexer инициализирует все зависимости для работы асинхронного индексатора: соединяется с Redis, создаёт сервер Asynq, регистрирует обработчики задач и запускает основной цикл обработки. Этот метод блокирует выполнение до завершения работы индексатора или возникновения ошибки.

Листинг A.9 – internal/app/app.go (фрагмент)

```
func (a *App) RunIndexer() error {
    deps, err := a.initCommonDependencies()
    if err != nil {
        return err
    }
    defer deps.DB.Close()

    log := deps.Log
    log.Info(
        "starting indexer",
        "redis", a.config.RedisURL, "concurrency", a.config.Queue.Concurrency,
    )

    // Asynq Server
    srv := queue.NewServer(
        a.config.RedisURL, 0, a.config.Queue.Concurrency,
    )

    // Services
    docService := document.New(
        deps.DocRepo, deps.ChunkRepo,
        deps.EmbedderClient, deps.Chunker,
        deps.MinIO,
        nil,
        log,
    )

    // Handlers
    docHandler := indexer_handlers.NewDocumentHandler(docService, log)

    // Router
    mux := indexer.NewRouter(docHandler)

    // Launching a indexer (blocks it)
    if err := srv.Run(mux); err != nil {
        return fmt.Errorf("asynq server failed: %w", err)
    }
    return nil
}
```

ПРИЛОЖЕНИЕ Б

ИСХОДНЫЙ КОД PYTHON-МИКРОСЕРВИСОВ

В этом разделе показаны ключевые компоненты двух Python-сервисов: эмбеддера (AI-обработка) и WEBUI (веб-интерфейс). Полный код доступен в репозитории GitHub (Приложение Е).

2.1 Точка входа эмбеддера (embedder/main.py)

Этот микросервис отвечает за все AI-операции: преобразование текста в векторы, ранжирование результатов поиска и извлечение ответов из документов. FastAPI обеспечивает высокую производительность HTTP-слоя, а механизм lifespan гарантирует, что модели трансформеров загружаются один раз при запуске и остаются в памяти для быстрого доступа.

Листинг Б.1 – embedder/main.py

```
import logging
from fastapi import FastAPI
from app.config import settings
from app.lifespan import lifespan
from app.logging_config import setup_logging
from app.routers import embed, rerank, qa, health
setup_logging(level=settings.log_level)
app = FastAPI(
    title="PDF Search Embedder Service",
    description=(
        "Microservice for generating text embeddings, reranking search results,"
        " and extracting answers using transformer models."
    ),
    version="1.0.0",
    lifespan=lifespan,
    docs_url="/docs",          # Swagger UI (default)
    redoc_url="/redoc",        # ReDoc documentation (default)
    openapi_url="/openapi.json" # OpenAPI schema (default)
)
app.include_router(embed)
app.include_router(rerank)
app.include_router(qa)
app.include_router(health)
@app.get("/", include_in_schema=False)
async def root():
    return {"message": "PDF Search Embedder Service", "docs": "/docs"}
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "app.main:app", host="0.0.0.0", port=settings.port, reload=False, log_level="info"
    )
```

2.2 Зависимости эмбеддера (requirements.txt)

Этот файл фиксирует версии всех библиотек, необходимых для работы AI-микросервиса. sentence-transformers преобразует текст в семантические векторы, torch выполняет вычисления на GPU/CPU, а fastapi с uvicorn обеспечивают быстрый HTTP-сервер.

Жёсткая привязка к версиям предотвращает конфликты при развёртывании и гарантирует, что все среды разработки используют одинаковые версии библиотек.

Листинг Б.2 – requirements.txt

```
fastapi==0.134.0
uvicorn==0.41.0
python-multipart==0.0.22
sentence-transformers==5.2.3
torch==2.10.0
transformers==5.2.0
pydantic==2.12.5
pydantic_settings
```

2.3 Фабрика Flask-приложения (WEBUI)

Фабричный паттерн позволяет создавать экземпляры приложения с разными конфигурациями для разработки, тестирования и продакшена. Этот подход упрощает управление зависимостями и делает код более модульным.

Приложение использует Redis для хранения сессий пользователей, blueprints для организации маршрутов по функциональным модулям и кастомные обработчики ошибок для удобной отладки.

Листинг Б.3 – WEBUI Flask Application Factory

```
def create_app(config_name='default'):  
    app = Flask(__name__)  
    # Load configuration  
    app.config.from_object(config_by_name[config_name])  
  
    # Extensions  
    swagger = Swagger(app)  
    Session(app)  
  
    # Register blueprints  
    from app.blueprints import health, auth, main  
    app.register_blueprint(health.bp)  
    app.register_blueprint(auth.bp)  
    app.register_blueprint(main.bp)  
  
    # Register error handlers  
    register_error_handlers(app)  
  
    return app
```

2.4 Клиент API с JWT-аутентификацией (WEBUI)

Этот клиент управляет всей коммуникацией между веб-интерфейсом и Go-сервером. Он автоматически извлекает JWT-токен из сессии пользователя и добавляет его в заголовки запросов, обеспечивая безопасную аутентификацию между микросервисами.

Клиент обрабатывает различные HTTP-методы (GET, POST, DELETE), проверяет статус ответов и корректно парсит JSON-данные. Если Go API возвращает ошибку, клиент логирует её для последующей отладки.

Листинг Б.4 – WEBUI API Client with JWT Authentication

```
def call_go_api(endpoint, method='GET', headers={}, **kwargs):
    """Send request to Go API and return (response_json, error_message)."""
    base_url = current_app.config['GO_API_BASE_URL'] # Import will be fixed
    url = f"{base_url}/{endpoint.lstrip('/')}"
    try:
        if method == 'GET':
            resp = requests.get(
                url, params=kwargs.get('params'), headers=headers, timeout=10
            )
        elif method == 'POST':
            resp = requests.post(
                url,
                data=kwargs.get('data'),
                json=kwargs.get('json'),
                files=kwargs.get('files'),
                headers=headers,
                timeout=3
            )
        elif method == 'DELETE':
            resp = requests.delete(url, headers=headers, timeout=10)
        else:
            return None, f'Unsupported method: {method}'

        resp.raise_for_status()
        if resp.content:
            try:
                return resp.json(), None
            except ValueError:
                logger.warning(
                    f"Non-JSON response from {url}: {resp.text[:200]}"
                )
                return resp.text, None
        return None, None

    except requests.exceptions.RequestException as e:
        logger.error(f"Error calling Go API at {url}: {e}")
        return None, str(e)

def call_go_api_auth(endpoint, method='GET', **kwargs):
    """Adds authorization token to the request."""
    token = session.get('token')
    headers = {'Authorization': f'Bearer {token}'} if token else {}
    return call_go_api(endpoint, method, headers, **kwargs)
```

ПРИЛОЖЕНИЕ В

КОНФИГУРАЦИИ DOCKER ДЛЯ РАЗВЁРТЫВАНИЯ

Эти файлы описывают, как все компоненты системы запускаются вместе в контейнерах Docker. Docker Compose координирует четыре основных сервиса, гарантируя их корректную совместную работу.

3.1 Основная оркестрация (docker-compose.yml)

Главный файл компоновки описывает четыре ключевых сервиса: AI-эмбеддер, индексатор документов, Go API Gateway и веб-интерфейс. Директива `depends_on` с условием `service_healthy` гарантирует, что сервисы запускаются в правильном порядке — например, база данных готова до попытки подключения к ней.

Проверки здоровья (`healthcheck`) позволяют Docker автоматически перезапускать неисправные контейнеры, повышая отказоустойчивость системы.

Листинг В.1 – *docker-compose.yml*

```
name: doc-archive
include:
  - infra/docker-compose.infra.yml
services:
  rag:
    build: ../embedder
    environment:
      EMBED_MODEL_NAME: \${EMBED_MODEL:-all-MiniLM-L6-v2}
      RERANK_MODEL_NAME: \${RERANK_MODEL:-cross-encoder/ms-marco-MiniLM-L-6-v2}
      READER_MODEL_NAME: \${READER_MODEL:-distilbert-base-cased-distilled-squad}
      HF_TOKEN: \${HF_TOKEN:-}
    ports:
      - "5001:5000"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
  indexer:
    build: ../server
    command: ["./indexer"]
    environment:
      DATABASE_URL: >
        postgres://\${POSTGRES_USER:-user}:\${POSTGRES_PASSWORD:-pass}
        @postgres:5432/\${POSTGRES_DB:-docdb}?sslmode=disable
      MINIO_ACCESS_KEY: \${MINIO_USER:-minioadmin}
      MINIO_SECRET_KEY: \${MINIO_PASSWORD:-minioadmin}
      MINIO_BUCKET: \${MINIO_BUCKET:-pdf-documents}
  api-gateway:
    build: ../server
    environment:
      DATABASE_URL: >
        postgres://\${POSTGRES_USER:-user}:\${POSTGRES_PASSWORD:-pass}
        @postgres:5432/\${POSTGRES_DB:-docdb}?sslmode=disable
      MINIO_ACCESS_KEY: \${MINIO_USER:-minioadmin}
      MINIO_SECRET_KEY: \${MINIO_PASSWORD:-minioadmin}
      MINIO_BUCKET: \${MINIO_BUCKET:-pdf-documents}
      SECRET_KEY: \${SECRET_KEY:-your-strong-secret-key}
    ports:
      - "8080:8080"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 10s
  webui:
    build: ../webui
    environment:
      SECRET_KEY: \${SECRET_KEY:-your-strong-secret-key}
      REDIS_URL: redis://redis:6379/0
    ports:
      - "5005:5000"
```


3.2 Инфраструктурные сервисы (docker-compose.infra.yml)

Эта конфигурация разворачивает три важнейших инфраструктурных компонента: PostgreSQL с поддержкой векторных операций, Redis для очередей и кэша, и MinIO для хранения файлов.

Сервис minio-init выполняет автоматическую настройку хранилища: создаёт бакет для документов, устанавливает политику автоматического удаления файлов через 7 дней и настраивает CORS для доступа из веб-браузера. Это позволяет начать работу с системой без ручной настройки.

Листинг B.2 – docker-compose.infra.yml

```
name: doc-archive-infra
services:
  postgres:
    image: pgvector/pgvector:pg15
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-user}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-pass}
      POSTGRES_DB: ${POSTGRES_DB:-docdb}
    ports: "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql
      - ../../migrations:/docker-entrypoint-initdb.d
    command: postgres -c shared_preload_libraries=vector
  redis:
    image: redis:7-alpine
    ports: "6379:6379"
    volumes:
      - redisdata:/data
    restart: unless-stopped
  minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_USER:-minioadmin}
      MINIO_ROOT_PASSWORD: ${MINIO_PASSWORD:-minioadmin}
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
      - miniodata:/data
  minio-init:
    image: minio/mc:latest
    depends_on:
      minio:
        condition: service_healthy
    volumes:
      - ./cors-config.json:/cors-config.json
      - ./lifecycle.json:/lifecycle.json
    entrypoint: minio-entrypoint.sh
```

Листинг В.3 – Entrypoint для бакета MinIO

```
mc alias set myminio http://minio:9000 \${MINIO_USER:-minioadmin} \${MINIO_PASSWORD:-minioadmin};
mc mb myminio/\${MINIO_BUCKET:-pdf-documents} --ignore-existing;
echo 'Bucket \${MINIO_BUCKET:-pdf-documents} created successfully';
mc ilm import myminio/pdf-documents < lifecycle.json
echo 'Lifecycle policy applied: expire objects after 7 days';
mc cors set myminio/\${MINIO_BUCKET} /cors-config.json;
echo 'CORS policy applied successfully.';
exit 0;
```

3.3 Переменные окружения (.env.example)

Этот файл-шаблон определяет все настройки, необходимые для запуска системы. Он включает параметры подключения к базам данных, секретные ключи для аутентификации и токены для загрузки AI-моделей.

Секретный ключ (SECRET_KEY) должен быть уникальным для каждого развёртывания (Листинг В.4) генерирует криптографически стойкий ключ.

Листинг В.4 – Генерация секретного ключа

```
openssl rand -base64 32
```

Токен Hugging Face (HF_TOKEN) позволяет загружать модели трансформеров, включая приватные репозитории.

Листинг В.5 – .env.example

```
ENV=production

POSTGRES_USER=user
POSTGRES_PASSWORD=pass
POSTGRES_DB=docdb

EMBED_MODEL=all-MiniLM-L6-v2
RERANK_MODEL=cross-encoder/ms-marco-MiniLM-L-6-v2
READER_MODEL=distilbert-base-cased-distilled-squad

SECRET_KEY=\$(openssl rand -base64 32)

HF_TOKEN=0000000000000000

GF_USER=admin
GF_PASSWORD=admin
```

ПРИЛОЖЕНИЕ Г

СКРИНШОТЫ ВЕБ-ИНТЕРФЕЙСА СИСТЕМЫ

В этом разделе представлены скриншоты основных страниц веб-интерфейса, которые демонстрируют, как пользователи взаимодействуют с системой. Каждый снимок показывает ключевые функции интерфейса — от регистрации до продвинутого поиска по документам.

```
~/s/doc-archive/deploy main !4 ?2 > docker-compose up -d
[+] up 8/8
✓ Container redis           Healthy           32.1s
✓ Container minio           Healthy           31.5s
✓ Container doc-archive-minio-init-1 Created           0.0ss
✓ Container indexer         Created           0.1ss
✓ Container api-gateway     Created           0.0ss
✓ Container web-ui          Created           0.1ss
✓ Container postgres        Healthy           31.1s
✓ Container rag              Healthy           31.1s
~/study/doc-archive/deploy main !4 ?2 > |
```

Рисунок Г.1 – Запуск всей системы через Docker Compose. Терминал показывает успешный старт всех четырёх основных сервисов и трёх инфраструктурных компонентов, подтверждая корректную оркестрацию контейнеров.

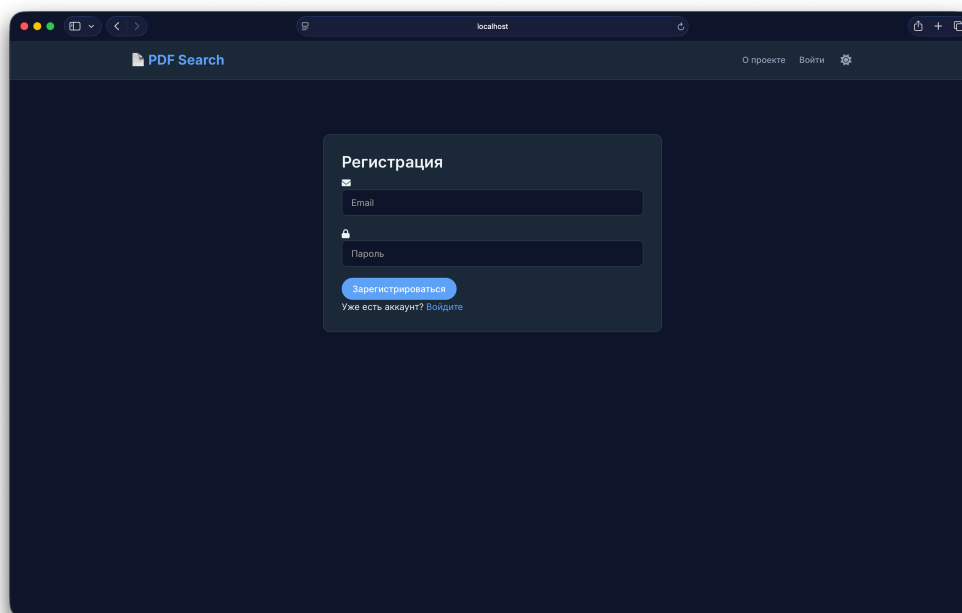


Рисунок Г.2 – Страница регистрации нового пользователя. Интерфейс включает проверку корректности email, требования к сложности пароля и мгновенную валидацию вводимых данных.

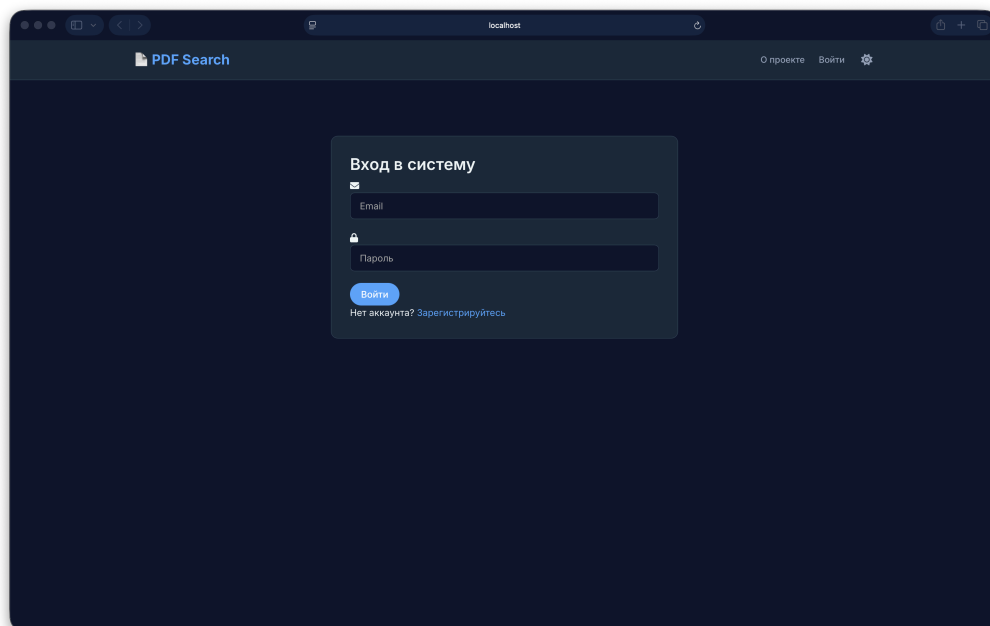


Рисунок Г.3 – Страница входа: форма аутентификации и доступ к восстановлению пароля

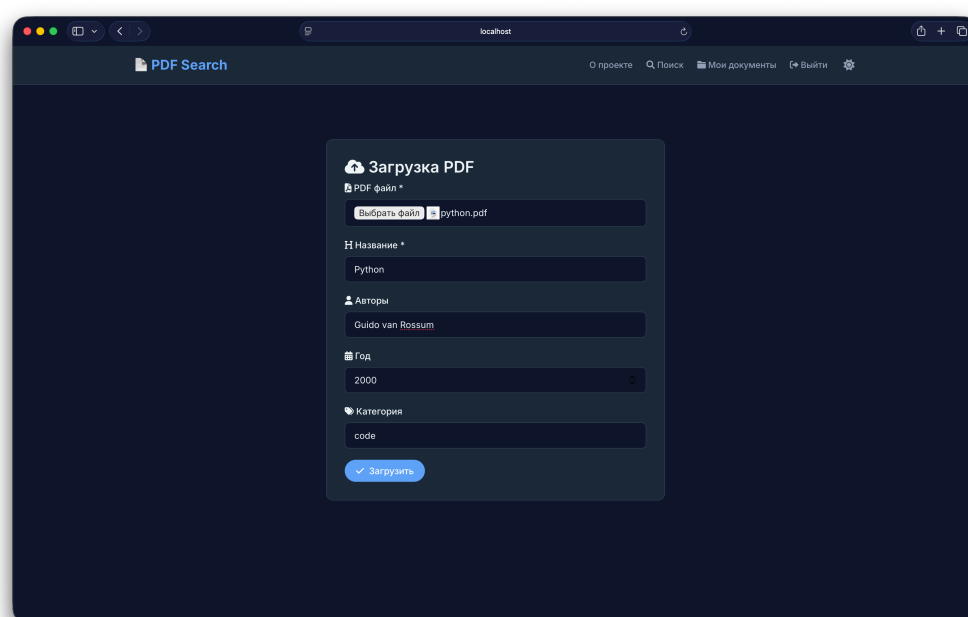


Рисунок Г.4 – Форма загрузки PDF: выбор файла, ввод метаданных и загрузка в хранилище

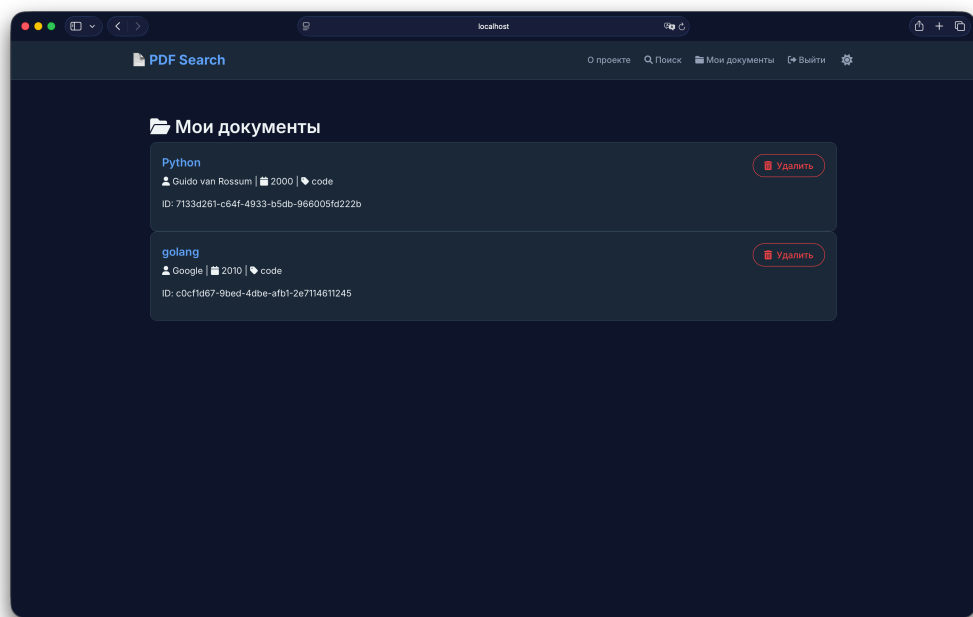


Рисунок Г.5 – Список документов пользователя в разделе «Мои документы» с превью и доступом к каждому документу

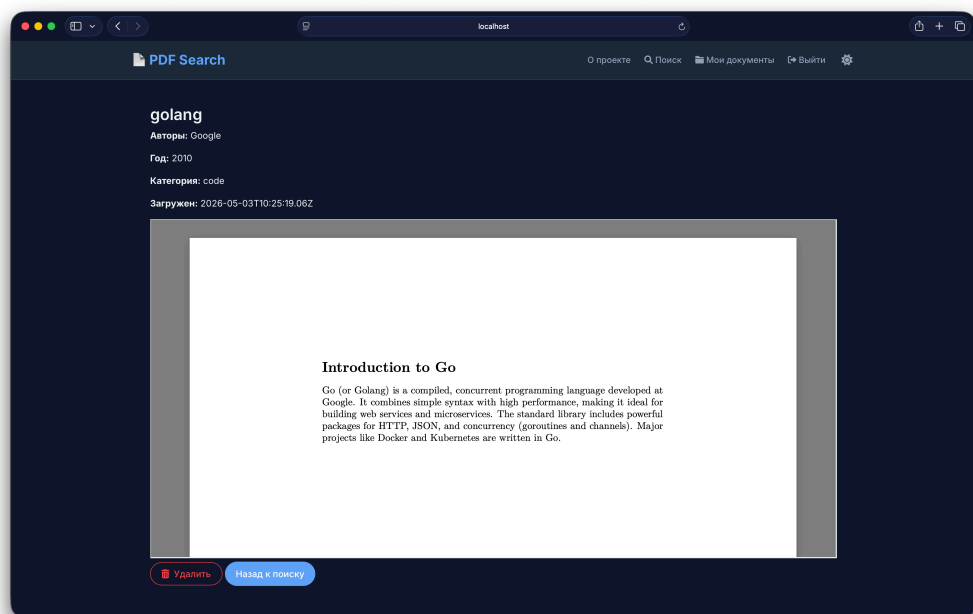


Рисунок Г.6 – Страница документа: подробности файла, метаданные и предпросмотр

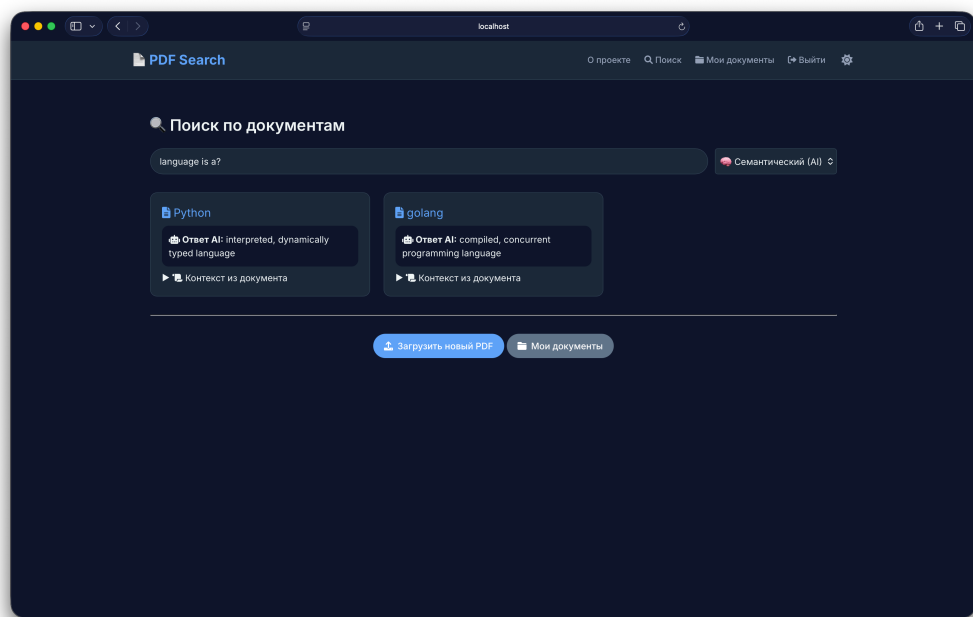


Рисунок Г.7 – Результаты поиска документов по запросу: отображение релевантных результатов

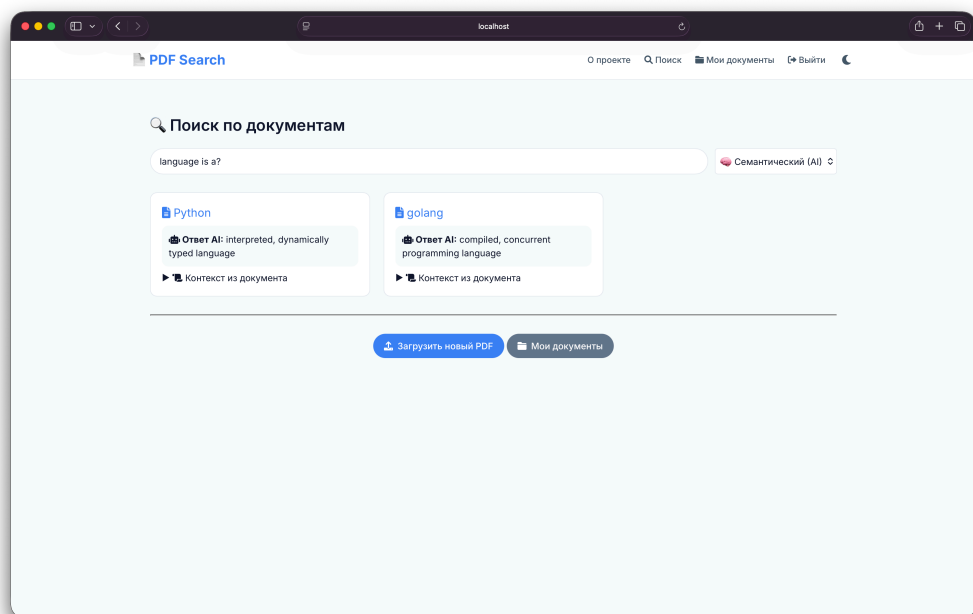


Рисунок Г.8 – Результаты поиска (светлая тема): визуализация тех же результатов в световой тематике

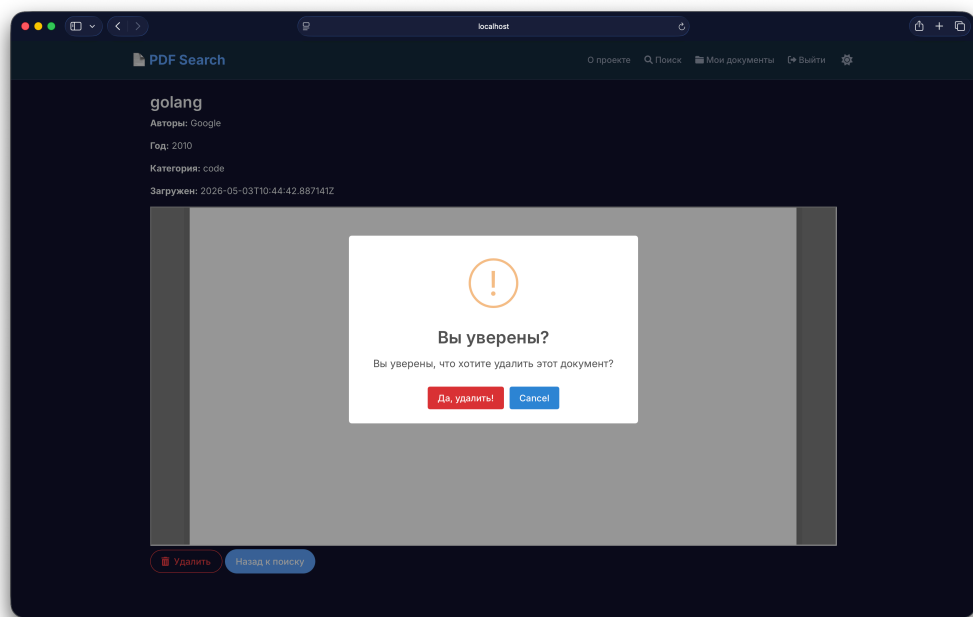


Рисунок Г.9 – Удаление документа: диалог подтверждения и удаление файла из хранилища

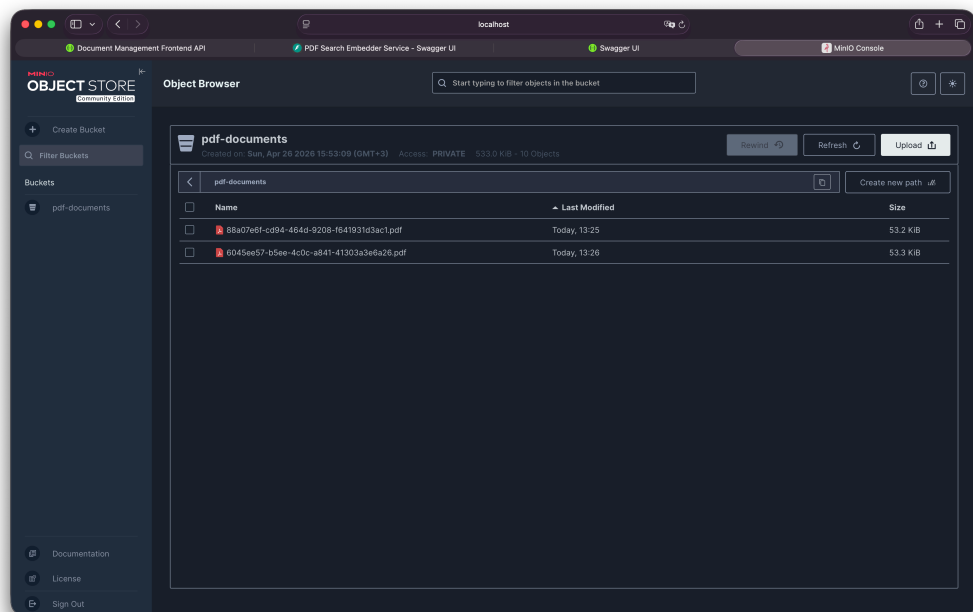


Рисунок Г.10 – Панель управления MinIO: обзор бакетов и объектов

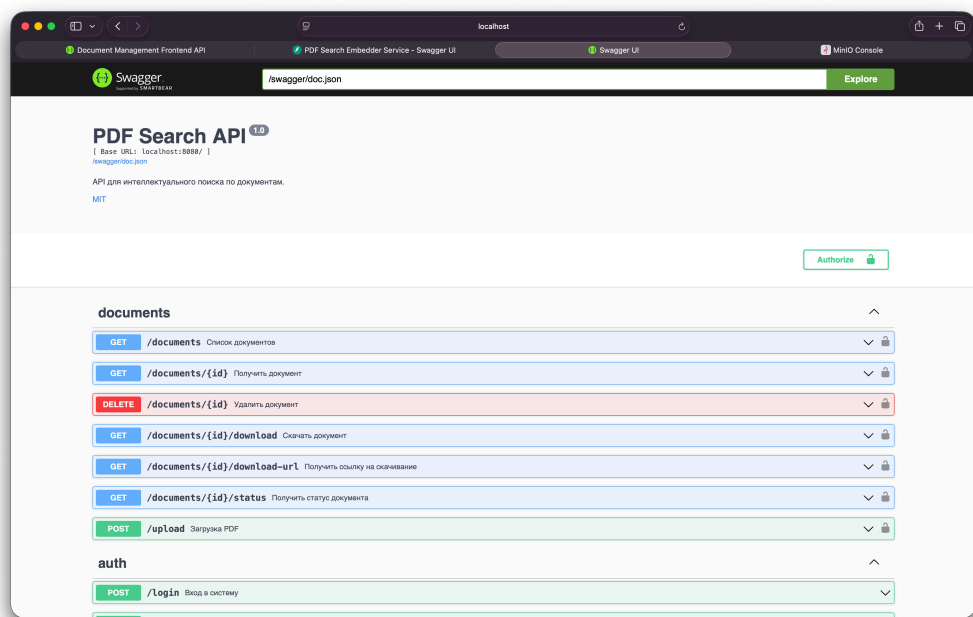


Рисунок Г.11 – Swagger для Go-серверов: интерактивная документация API

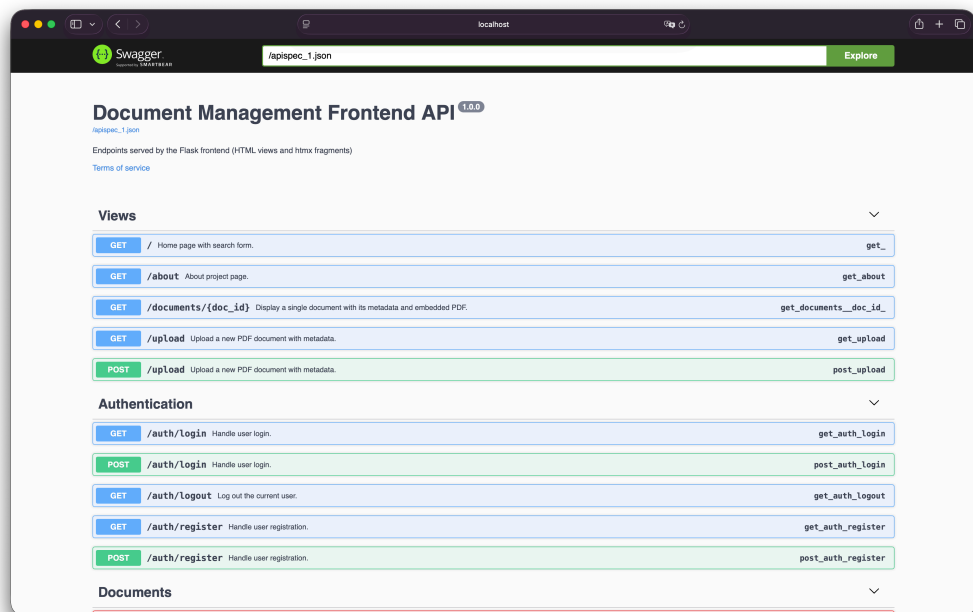


Рисунок Г.12 – Swagger UI: веб-интерфейс для взаимодействия с API проекта

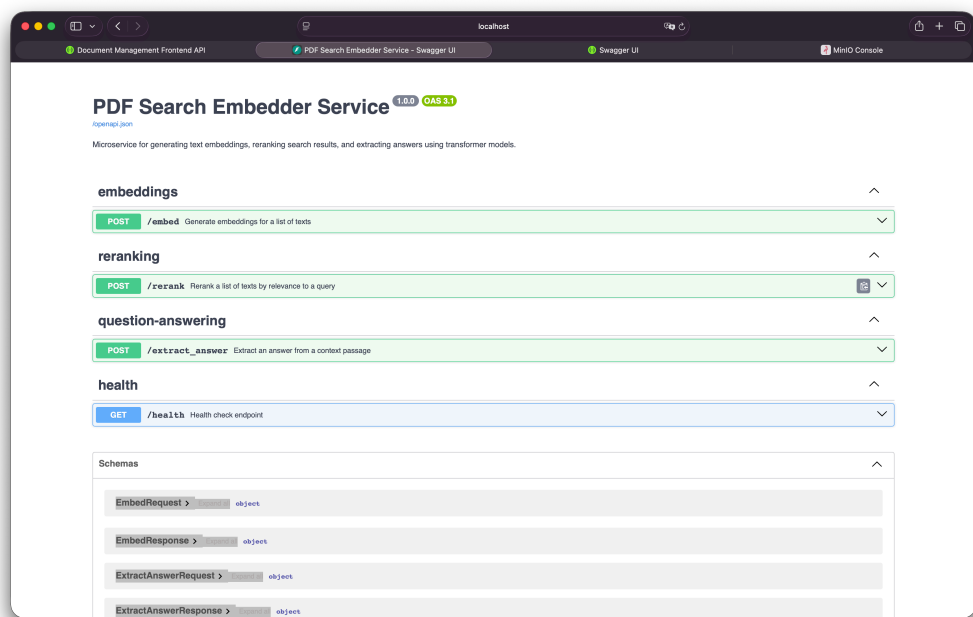


Рисунок Г.13 – Swagger-документация для ИИ-микросервиса: эндпоинты и интеграции

```
~/study/doc-archive/server main !7 ?17 > go test -count=1 ./... 2>&1 | grep -E '^(ok|FAIL)'
ok      github.com/AndB0ndar/doc-archive/internal/domain      0.859s
ok      github.com/AndB0ndar/doc-archive/internal/service/chunker 0.235s
ok      github.com/AndB0ndar/doc-archive/internal/service/document 0.325s
ok      github.com/AndB0ndar/doc-archive/internal/service/search 0.773s
ok      github.com/AndB0ndar/doc-archive/internal/transport/http/handlers 0.634s
ok      github.com/AndB0ndar/doc-archive/internal/transport/http/middleware 0.476s
ok      github.com/AndB0ndar/doc-archive/pkg/jwt              0.865s
~/study/doc-archive/server main !7 ?17 >
```

Рисунок Г.14 – Результаты тестирования Go-проекта и покрытие кода

ПРИЛОЖЕНИЕ Д

SQL-МИГРАЦИИ И ЗАПРОСЫ К БАЗЕ ДАННЫХ

В этом разделе представлены SQL-скрипты, которые создают структуру базы данных и реализуют ключевые функции поиска. PostgreSQL с расширениями обеспечивает как традиционный полнотекстовый поиск, так и современный семантический поиск по векторам.

5.1 Начальная миграция базы данных

Этот скрипт создаёт всю структуру базы данных с нуля. Он включает три важнейших расширения: `uuid-ossr` для генерации уникальных идентификаторов, `pg_trgm` для быстрого поиска по тексту и `vector` для работы с эмбедингами.

Таблицы `users`, `documents` и `chunks` образуют основу системы. Индексы HNSW особенно важны — они ускоряют поиск по векторам в сотни раз, делая семантический поиск практически мгновенным.

Листинг Д.1 – Создание таблиц

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email TEXT NOT NULL UNIQUE, password_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TYPE document_status AS ENUM ('pending', 'processing', 'done', 'error');
CREATE TABLE documents (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title TEXT NOT NULL, authors TEXT, year INTEGER, category TEXT,
  file_path TEXT NOT NULL, file_size BIGINT NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
  CONSTRAINT documents_status_check CHECK (status IN ('pending', 'processing', 'done', 'error'))
);
CREATE TABLE chunks (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  content TEXT NOT NULL, embedding vector(384), index INTEGER NOT NULL,
  UNIQUE(document_id, index)
);
```

Листинг Д.2 – Добавление расширений

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";  
CREATE EXTENSION IF NOT EXISTS "pg_trgm";  
CREATE EXTENSION IF NOT EXISTS "vector";
```

Листинг Д.3 – Создание индексов

```
CREATE INDEX idx_documents_user_id ON documents(user_id);  
CREATE INDEX idx_documents_status ON documents(status);  
CREATE INDEX idx_chunks_document_id ON chunks(document_id);  
CREATE INDEX idx_chunks_content_trgm ON chunks  
    USING gin (content gin_trgm_ops);  
CREATE INDEX idx_chunks_embedding ON chunks  
    USING hnsf (embedding vector_cosine_ops);
```

5.2 Полнотекстовый поиск по триграммам

Этот запрос реализует «гибкий» поиск, который находит документы даже при опечатках или неполном совпадении слов. Функция `similarity` вычисляет коэффициент схожести между запросом пользователя и текстом документа на основе триграмм — последовательностей из трёх символов.

Такой подход особенно полезен для поиска по техническим терминам, фамилиям авторов или специфической терминологии, где пользователь может допустить ошибку в написании.

Листинг Д.4 – Полнотекстовый поиск

```
SELECT  
    c.id, c.document_id, c.content,  
    similarity(c.content, \ $1) AS confidence  
FROM chunks c  
JOIN documents d ON c.document_id = d.id  
WHERE d.user_id = \ $2  
ORDER BY similarity(c.content, \ $1) DESC  
LIMIT 20;
```

5.3 Семантический поиск по векторам

Этот запрос выполняет поиск не по точному совпадению слов, а по смысловому сходству. Оператор «`<=>`» вычисляет косинусное расстояние между вектором запроса пользователя и векторами документов, возвращая наиболее релевантные результаты.

Индекс HNSW (Hierarchical Navigable Small World) делает этот поиск чрезвычайно быстрым — даже при миллионах векторов в базе данных. Это позволяет системе находить документы по смыслу, а не только по ключевым словам.

Листинг Д.5 – Семантический поиск

```
SELECT
    c.id, c.document_id, c.content,
    1 - (c.embedding <=> \ $1) AS confidence
FROM chunks c
JOIN documents d ON c.document_id = d.id
WHERE c.embedding IS NOT NULL AND d.user_id = \ $2
ORDER BY c.embedding <=> \ $1
LIMIT 20
```

ПРИЛОЖЕНИЕ Е

ССЫЛКА НА РЕПОЗИТОРИЙ ПРОЕКТА

Проект размещён на платформе GitHub: <https://github.com/AndB0ndar/doc-archive>.

В репозитории содержится полный исходный код, документация и инструкции по развёртыванию.